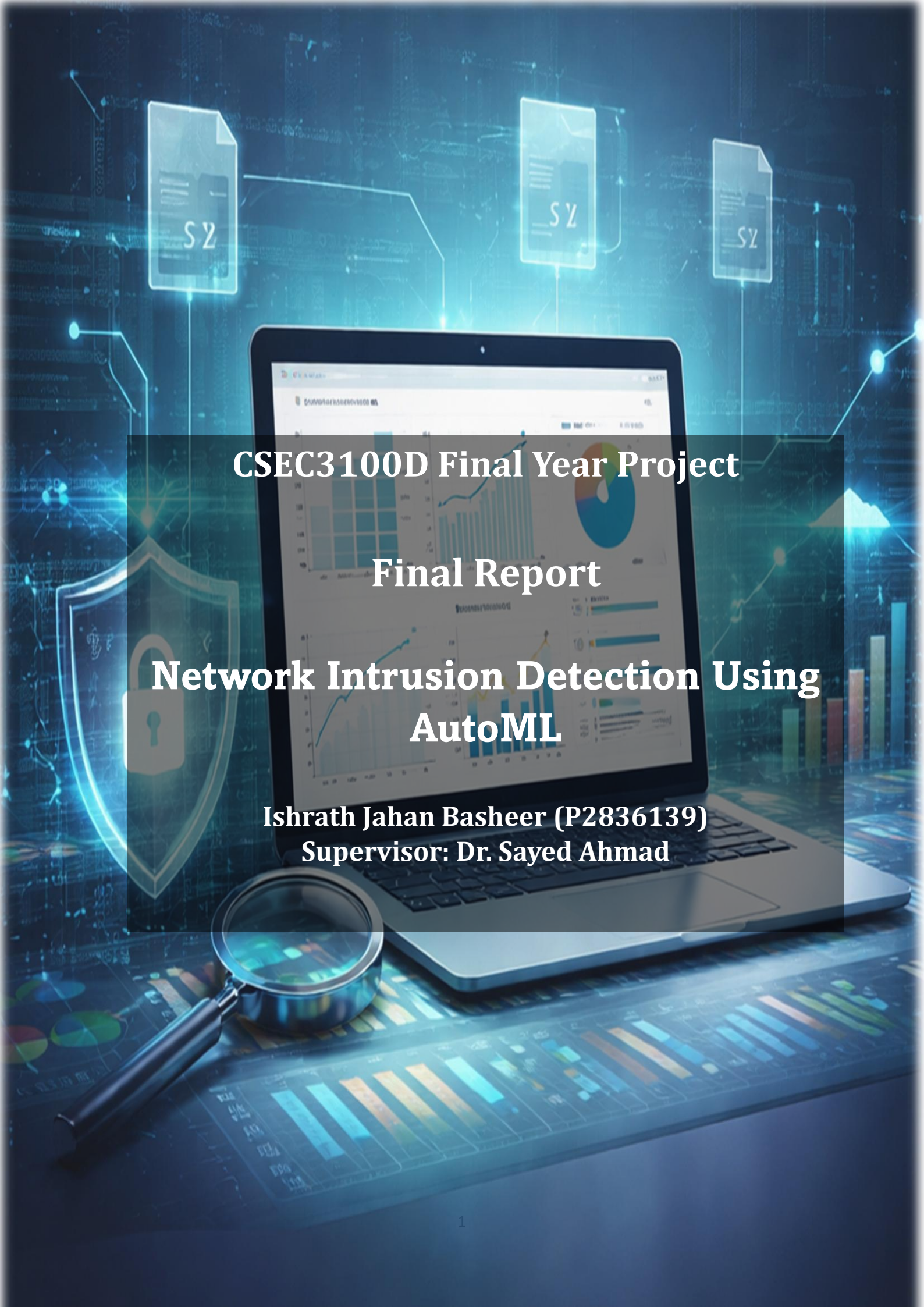


CSEC3100D Final Year Project

Final Report

Network Intrusion Detection Using AutoML

Ishrath Jahan Basheer (P2836139)
Supervisor: Dr. Sayed Ahmad

ACKNOWLEDGMENTS

I would first of all like to thank my supervisor, Dr. Sayed Ahmad, who assisted me throughout my project, had the opportunity to give me valuable suggestions, and supported me in terms of guidance and encouragement. His contribution has played an important role in developing the scope of this project, the methodology of evaluation, and the general direction in which the research would be heading.

De Montfort University Dubai would also be appreciated to have been provided the learning facilities and computing facilities that facilitated the performance of this work successfully.

Lastly, I acknowledge my sincere gratitude to my family for being a constant source of support throughout my project.

DECLARATION

In keeping with De Montfort University Dubai academic regulations, I pledge that this work is my own and that I have not received inappropriate assistance in its preparation.

I further declare that any published or unpublished material, if used, has been acknowledged.

Student Name: Ishrath Jahan Basheer Signature: Ishrath

Date of Submission: 9 May 2026

Table of Contents

Abstract 5
Standards, Constraints and Risk Assessment 6
1.1 Project Overview 8
1.2 Aims and Objectives 8
2. Background and Literature Review 9
3. Requirements Specification 14
3.4 Indicative Test Plan 18
4. System Design and Architecture 23
4.3 Data Flow 25
5. Methodology and Implementation 27
5.3 Dataset Loading and Cleaning 28
5.4 Label Encoding and Feature/Label Split 30
5.5 Train/Test Split and Scaling 30
5.6 Logistic Regression (Baseline 1) 31
5.7 Random Forest (Baseline 2) 33
5.8 H2O AutoML 36
5.9 SHAP Explainability 39
5.10 Prediction Demo 41
6. Evaluation and Results 42
6.2 Confusion Matrix Analysis 43
Interactive Dashboard - Streamlit Application 44
6.5 Answer to the Research Questions 49
7. Discussion and Critical Analysis 50
8. Global and Societal Impact 52
9. Project Management and Reflection 53
10. Conclusion and Future Work 54
11. References 56
12. Appendices 59

Abstract

This project examines if an accurate network intrusion detection system (NIDS) can be developed with low manpower by implementing an AutoML (Automate Machine Learning). A binary classifier was constructed using CICIDS2017 dataset to classify the normal (BENIGN) and PortScan attacks. Scikit-learn was used to train two manually constructed baseline models (Logistic Regression and Random Forest) and then compare those models with one trained automatically in 600 seconds or less using H2O AutoML.

All three models were tested with the same test set of previously unseen data with the metrics of precision, recall, F1-scor, ROC-AUC, and confusion matrix. The best model was used for SHAP analysis to find the most influential features in the network flow. All three models made near-perfect classification with only 5 misclassifications in 57,220 test records for both Random Forest and H2O AutoML (GBM) compared to 50 misclassifications for Logistic Regression. This shows that AutoML can yield comparable intrusion detection models without algorithm selection and hyperparameter tuning. The report treats all aspects of a project from literature review and requirements to implementation, evaluation, explainability, and critical reflection.

Standards, Constraints and Risk Assessment

Standards

The project follows the IEEE 830 conventions for requirements specification, which is each functional requirement is expressed in a verifiable "The system shall..." style with prioritisation using MoSCoW. All library versions are logged and reproducibility is enforced by a fixed random seed (random_state=42). The CICIDS2017 data is provided for non-commercial, academic research only, and in accordance with the terms of use by the Canadian Institute for Cybersecurity. All the sources are referenced in Harvard style using a DOI and date of access.

Constraints

Hardware: All experiments ran on a standard student laptop with no GPU, which limited the AutoML budget to 600 seconds.

Dataset: CICIDS2017 is a static, pre-captured dataset from a simulated 2017 environment. It does not reflect current attack patterns or real enterprise traffic, so model performance is evaluated against this historical benchmark only.

Scope: The project is restricted to binary classification (PortScan vs. BENIGN) using a single-day CICIDS2017 subset. Multi-class detection across all attack types was intentionally excluded to keep the comparison methodologically focused.

Tool Constraint: H2O AutoML requires a Java Runtime Environment (JDK 25.0.3 used in this project) to initialise its local cluster. This dependency introduces a setup step that is not required by scikit-learn, and any environment without Java installed would prevent H2O from running. TPOT was identified as a fallback AutoML strategy (FR9) to mitigate this risk.

Time: Completed within the 2025–26 academic year as part of CSEC3100D. The fixed schedule influenced the decision to use a single-day CSV rather than the full multi-day CICIDS2017 corpus.

Risk Assessment

Risk ID	Risk Description	Likelihood	Impact	Mitigation	Status
R1	H2O fails to initialise missing Java	Medium	High	TPOT as backup (FR9); JDK installed and verified	solved
R2	Data leakage from fitting scaler on full dataset	Medium	High	StandardScaler fitted only on X_train; .transform() on X_test (FR6)	solved
R3	Dataset artefacts cause performance metrics to be inflated	High	Medium	SHAP as sanity check: top features confirmed from PortScan behaviour (FR13	solved
R4	AutoML exceeds hardware time budget	Medium	Medium	Hard cap at max_runtime_secs=600; tested before final run	solved
R5	Class imbalance leads to misjudging accuracy	Medium	Medium	Stratified split (stratify=y); F1 and ROC-AUC reported with accuracy (FR10)	solved
R6	Notebook gives different results on re-run	Low	High	random_state=42 throughout; reproducibility verified by TC17	solved
R7	Silently crashes pipeline if invalid input in CSV file; No error in pipeline if CSV is missing with Label column (FR16, TC15)	Low	Medium	try/except around label encoding; descriptive error if Label column missing	solved

1.1 Project Overview

Networks are an increasingly vulnerable target today. Port scanning, brute force or Denial of Service (DoS) floods, are all ways in which organisations of all sizes are finding it difficult to keep their systems secure. The Network Intrusion Detection System (NIDS) is one of the crucial defenses against such attacks: it continuously watches traffic and alerts to any suspicious activity.

Rule-based NIDS are based on the idea of comparing packets against a database of known attack signatures, like Suricata and Snort. They work well with known attacks, but not with new attacks or attacks that are slight variations on known attacks. To overcome this, machine learning systems, such as a NIDS, can learn patterns directly from labeled traffic data and are able to detect known and new attacks without relying on hand-crafted rules.

However, the difficulty is that it is not easy to develop a successful ML model. You need to choose the appropriate algorithm, meticulously tune your hyperparameters, deal with noisy real-world data, and make sure you don't have issues such as data leakage things that many small organisations do not necessarily have on staff.

That's where Automated Machine Learning (AutoML) comes in. Many of the hyperparameters and algorithms are explored automatically in frameworks, within a given time limit, and the best performing model is returned, minimizing the manual effort. The key research question that this project seeks to answer is: Is it possible to create a network intrusion detection model with AutoML that achieves similar performance to a manually built model, but in significantly less time and with less expertise?

To do this, I used the open-source dataset CICIDS2017 and evaluated two handcrafted classifiers Logistic Regression and Random Forest with an automatically generated model from H2O AutoML. To overcome another criticism of ensemble models, namely that they are "black boxes" that make difficult to explain decisions, SHAP analysis was also used when the best model was selected.

1.2 Aims and Objectives

This project aims to implement, train and test an AutoML binary network intrusion detection system with CICIDS2017 dataset and benchmark with the existing machine learning baselines.

The specific objectives are:

- To search the related literature on ML-based intrusion detection and AutoML to find what has been accomplished and the areas that are lacking.

- To obtain and preprocess the CICIDS2017 dataset, there are many issues to be addressed, including how to deal with infinite values, missing values, label encoding, and data leakage prevention.

- To Train and evaluate Logistic Regression and Random Forest as manually-built baseline classifiers.

- To execute H2O AutoML for a fixed duration of 600 seconds and let the models select the best model.

- To compare all three models on the same held-out test set, with the use of standard classification metrics.

- To apply SHAP analysis to the best model to identify the most influential features and validate that the model learned genuine attack patterns.

- To evaluate the results, limitations and the project process in general.

1.3 Research Questions

Three research questions guided this project:

RQ1: AutoML versus Logistic Regression and Random Forest in binary attack detection on CICIDS2017: When comparing AutoML with Logistic Regression and Random Forest, does AutoML achieve better performance, given a fixed budget amount and preprocessing?

RQ2: How does the compromising aspect of predictive performance versus practical costs of using AutoML (training time, model complexity) over baselines?

RQ3: Are feature importance or SHAP methods able to identify plausible features of networks (network flows)? And are they useful in detecting artefact-influenced learnin

2. Background and Literature Review

Keywords: Network intrusion detection system (NIDS), CICIDS2017, flow features, supervised learning, Logistic Regression, Random Forest, AutoML, H2O AutoML, TPOT, evaluation metrics, class imbalance, data leakage, explainable AI (XAI), feature importance, SHAP.

Introduction and purpose

Networked systems represent one of the primary targets of the cyberattacks, and intrusion detection system (IDS) is the commonly used tool that organisations employ to track suspicious behaviour. Conventional IDS methods are based on signature detection and predefined rules. Although these are effective in dealing with known threats, they fail to keepup with new or changing attack types. To overcome this shortcoming, more recent studies are using machine learning (ML) methods to identify intrusions using trends in network traffic data. Such methods enhance the element of flexibility and decreased the maintenance of rules manually (Pinto et al., 2023; Rahman et al., 2025).

This project follows this path and uses a supervised learning approach to the CICIDS2017 dataset, which is one of the most commonly used benchmark datasets of labelled network flow data (Sharafaldin, Habibi Lashkari and Ghorbani, 2018).

Scope

This review concentrates on network intrusion detection based on machine learning through flow based supervised learning. The scope is described as the following:

Domain: Network Intrusion Detection Systems (NIDS), but not host-based IDS.
Dataset: CICIDS 2017, its strengths, and weaknesses (Sharafaldin et al., 2018; Engelen et al., 2021; Rosay et al., 2022).
Formulation of the problem: Binary classification (Attack vs Normal) in order to make it feasible.
Methods: Compared to baseline ML models (Logistic Regression, Random Forest), such approaches as AutoML (Maseer et al., 2021; Khan et al., 2023; Yang and Shami, 2024) should be compared.
Evaluation: Indicators that are used where there is imbalanced data (Precision, Recall, F1- score, ROC-AUC) and confusion matrices.
Explainabilty: SHAP based explanation and feature significance (Lundberg and Lee, 2017; Gaspar et al., 2024; Mohale and Obagbuwa, 2025)

The main data set that will be used in this research is the CICIDS2017 data set created by the Canadian Institute for Cybersecurity (Canadian Institute for Cybersecurity, 2017)

Literature search strategy and sources

The review has been conducted on the basis of 16 academic sources, such as peer-reviewed journal articles and conference papers.

IEEE Xplore, ACM Digital Library, Scopus and Google Scholar were used to identify the sources. Publishers that were used to obtain full-text papers included IEEE, ACM, ScienceDirect, Frontiers, MDPI, and SciTePress.

The search terms were also grouped and combined by using
Boolean operators: IDS/NIDS: "intrusion detection", "network intrusion detection."
Dataset: "CICIDS2017", "flow features".
Modelling: "random forest", "logistic regression", "evaluation metrics". AutoML: “H2O AutoML”, “TPOT”
Explainability: "SHAP", "XAI", "feature importance".

The review is primarily focusing on studies that are published during 2021-2026, with older works considered where necessary (e.g. SHAP and CICIDS2017 dataset). (Lundberg and Lee, 2017; Sharafaldin, Habibi Lashkari and Ghorbani, 2018).

ML-based IDS: Advantages and Practical Limitations.

Machine learning-based IDS has better detection functionality as it learns more complicated patterns with the help of large data. Pinto et al. (2023) emphasise the usefulness of ML in detecting various types of attacks, especially with critical infrastructure. In the same way, Rahman et al. (2025) indicate that ML enhances dynamic IoT environments.

These advantages are subject to notable constraints; however Rahman et al. (2025) also identify the lack of resources and fluctuations in network traffic as a challenge. Compared to optimism provided by surveys, there is a more pessimistic perspective of empirical studies. In particular, Chua and Salam (2023) show that the performance of models can differ significantly when evaluated on other stages of data.

Relevance to this project's findings: Logistic Regression, Random Forest and H2O AutoML (GBM) models in this project had near perfect top line metrics on CICIDS2017. When looking at the raw confusion matrices, however, it was found that Logistic Regression was the model with 50 misclassifications while both tree-based models only had 5. This directly supports Chua and Salam's (2023) point: surface-level accuracy figures can obscure meaningful differences that only appear on closer inspection.

CICIDS2017 Dataset: Strengths and Limitations.

CICIDS2017 is popular because it has realistic traffic scenarios and attacks that are labelled (Sharafaldin et al., 2018). There are however a number of studies which point to major limitations.

Engelen et al. (2021) mention the problems connected to data quality, such as inconsistencies in flow generation and feature extraction. According to them, models can pick up artefacts of datasets and not real attack patterns. In a similar manner, Rosay et al. (2022) define the issues in flow construction and suggest flow improvements.

Relevance to this project's findings: The concern which was raised by Engelen et al. (2021) was addressed in two ways.First, strict leakage prevention was enforced and the StandardScaler was only fitted on training data and second, SHAP analysis served as a sanity check: the top-ranked features all carry clear cybersecurity meaning for PortScan behaviour, with no artefact-like or uninterpretable features appearing in the top 15. This directly supports the conclusion that the model learned meaningful patterns instead of the dataset noise.

Baseline ML and Evaluation Problems.

One of the work points of this project is to compare the AutoML with the conventional ML baselines. The significance of the baseline models lies in the fact that they can present a clear reference point: in case AutoML is complementary, or even inconsistent, a less

complicated baseline can be a more effective option in the context of the realistic IDS pipeline.

Maseer et al. (2021) highlight that discrepancies in the assessment procedures may have a considerable impact. On the same note, Kilincer et al. (2021) demonstrate that dataset characteristics and preprocessing techniques are important in performance.

Class imbalance is one of the problems with the IDS evaluation. The precision in itself can be deceptive since classifiers can be biased to the majority (Pinto et al., 2023; Rahman et al., 7 2025). Besides, as Engelen et al. (2021) point out, aggregated metrics may conceal the poor performance of a certain class.

One more serious concern is the data leakage. It has been shown that unrealistic performance can be caused by improper preprocessing or random splitting (Engelen et al., 2021; Rosay et al., 2022).

Relevance to this project's findings: This project addressed all three concerns. A fixed random_state=42 was used across all models for reproducibility, StandardScaler was fitted on training data only, and data cleaning was applied before the train/test split. The fact that even Logistic Regression achieved a ROC-AUC of 0.9999 confirms that the high performance is a genuine property of the data not a preprocessing artefact.

AutoML Intrusion Detection: why and where it is constructive and where it is misleading.

AutoML minimizes the amount of manual work by automating the choice of models and hyperparameter optimization. Yang and Shami (2024) present an AutoML system of autonomous intrusion detection, whereas Khan et al. (2023) show better results with ensemble-based AutoML solutions.

However, there are also challenges presented by AutoML. It is computationally intensive and can create complicated models which are hard to interpret.

Relevance to this project's findings: H2O AutoML explored multiple model configurations across five algorithm families within the 600-second budget and automatically selected GBM_1 as the best model. This model matched Random Forest in accuracy (5 total errors) without requiring any manual algorithm selection. The GBM_1 model contains 135 trees with a mean depth of 14.88 moderately complex which confirms the literature's observation that AutoML tends to favour more complex solutions. The leaderboard confirmed that GBM outperformed all other families including Deep Learning and GLM.

Explainability in IDS: usefulness and limitations.

Explainability is also beginning to be regarded as a security requirement because an analyst is frequently required to know why traffic was flagged. Mohale and Obagbuwa (2025) describe the implementation of explainable AI in ML-based IDS and suggest that the matter of transparency can enhance trust and aid in decision-making.

The paper is directly related to the plan of this project because Gaspar, Silva and Silva (2024) compare LIME and SHAP to the context of IDS, which will also involve explanations. Another applied example described by Sharma et al. (2024) is the case of LIME and SHAP to understand a deep-learning-based IoT network IDS. Much of their datasets and scale is out of the scope of this project binary CICIDS2017 environment, but the support on their argument is that explanation techniques can assist identify influential features and convey model behaviour.

Relevance to this project's findings: SHAP analysis on the Random Forest model confirmed that the top-ranked features including Destination Port, Flow Duration, Total Forward/Backward Packets, and RST Flag Count all correspond directly to known PortScan traffic characteristics. No unexpected or artefact-like features appeared in the top 15, supporting Mohale and Obagbuwa's (2025) argument that explainability can enhance trust by making the model's reasoning transparent.

Summary and research gap

The literature consistently supports ML for network intrusion detection, but three challenges recur across sources: dataset quality and artefacts (Engelen et al., 2021; Rosay et al., 2022), evaluation methodology and leakage (Maseer et al., 2021; Chua and Salam, 2023), and the interpretability deficit of high-performing ensemble models (Gaspar et al., 2024; Mohale and Obagbuwa, 2025). AutoML offers a promising route to reducing manual effort (Khan et al., 2023; Yang and Shami, 2024), but requires careful benchmarking to be credible.

Research gap:

Few studies directly compare AutoML frameworks against manual baselines under controlled, leakage-free conditions on CICIDS2017 while simultaneously addressing dataset artefacts and providing explainability analysis. Existing work either benchmarks performance without explainability (Maseer et al., 2021; Kilincer et al., 2021), or applies AutoML without rigorous baseline comparison (Khan et al., 2023; Yang and Shami, 2024). This project addresses that gap by combining a reproducible evaluation pipeline with SHAP-based transparency to produce results that are both defensible and interpretable

3. Requirements Specification

3.1 Overview

System requirements were defined based on the project contract .They are divided into functional requirements (what the system must do) and non-functional requirements (how it must behave).

What are in the Scope:

- Offline batch classification experiments on the CICIDS2017 CSV dataset
- Binary classification of PortScan attack flows vs. BENIGN traffic
- Manual baseline models (Logistic Regression, Random Forest) and H2O AutoML comparison
- Standard evaluation metrics: Confusion Matrix, Precision, Recall, F1, ROC-AUC
- Model explainability by SHAP feature importance analysis
- Export of all results, plots, and run settings for reproducibility

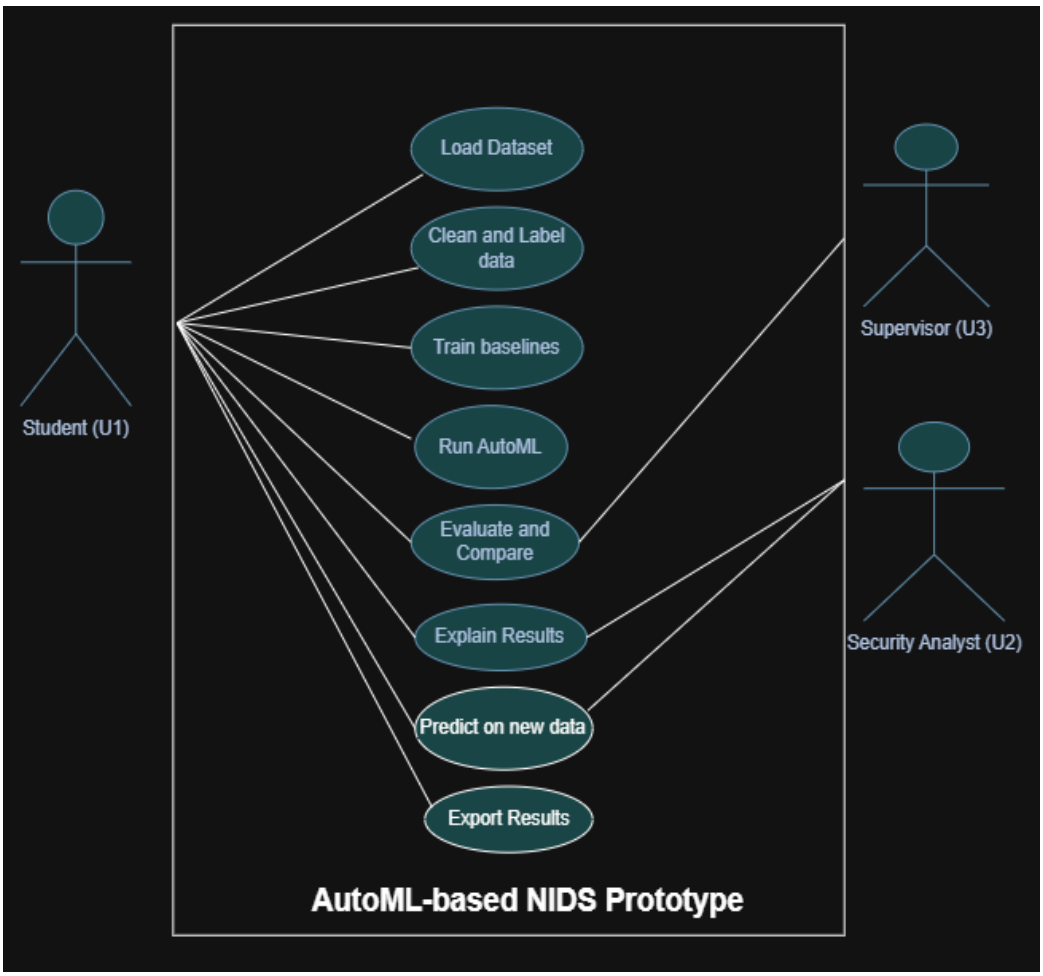
What are out of the Scope:

- Deployment on a live enterprise network
- Live traffic capture or new dataset generation
- Deep packet inspection or traffic decryption.

System users and stakeholders

User ID	User	Role in this project
U1	Student (me)	Manages the workflow related to the use of the notebook, trains models, with their analysis, and prepares a report.
U2	Security analyst (intended user)	Uses the trained model to classify the new traffic and explores the model explanations.
U3	Supervisor/Marker	Checks methodology, confirms compliance with the project contract and compliance with reproducibility.

Use case Diagram



Use Cases summary

Use Case	Actor	Goal
UC1 Import dataset	U1	Load CICIDS2017 CSV and verify data loaded correctly
UC2 Clean & label data	U1	Handle missing/inf values and map labels to binary
UC3 Split data	U1	Divide data reproducibly without leakage
UC4 Train baselines	U1	Train Logistic Regression and Random Forest
UC5 Run AutoML	U1	Find best model automatically using H2O AutoML
UC6 Evaluate models	U1, U3	Compare all models using standard metrics
UC7 Explain results	U1, U2	Generate feature importance and SHAP plots
UC8 Predict new data	U2	Classify unseen flows as BENIGN or ATTACK
UC9 Export results	U1	Save metrics, plots and run settings for reproducibility

*The full use cases are uploaded in the student cartridge.

Functional requirements

ID	Functional requirement	Priority
FR1	The system shall allow importing a CSV file containing CICIDS2017 data, display a live preview, and show a data summary.	Must
FR2	The system shall display the label distribution (Attack vs Normal) with row counts.	Must
FR3	The system shall detect all infinite and invalid values, remove affected rows, and record the count of alterations made.	Must
FR4	The system shall map all target labels to binary values (0 = BENIGN, 1 = ATTACK).	Must

FR5	The system shall split the dataset into training and test sets using a fixed random seed (random_state=42) with an 80/20 ratio.	Must
FR6	The system shall prevent data leakage by fitting all preprocessing steps exclusively on the training set.	Must
FR7	The system shall train Logistic Regression and Random Forest models as baseline classifiers on the training set.	Must
FR8	The system shall execute H2O AutoML with a user-defined time budget, save the full leaderboard, and store the best model.	Must
FR9	The system should execute TPOT as an alternative AutoML strategy if H2O AutoML fails to initialise.	Should
FR10	The system shall evaluate all models using Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix on the held-out test set.	Must
FR11	The system shall present a formatted comparison table of all model evaluation metrics	Must
FR12	The system shall compute and display global feature importance rankings for the best-performing model.	Must
FR13	The system should compute SHAP explanations for a sample of test instances and generate a beeswarm summary plot.	Should
FR14	The system shall generate BENIGN/ATTACK predictions with probability scores for new unseen flow data supplied as a CSV.	Must
FR15	The system shall export all evaluation plots, metrics tables, and run settings to an output folder for reproducibility.	Must
FR16	The system shall display an informative error message and halt execution if required columns are missing or input files are invalid.	Must

ID	Non-Functional Requirement	Category	Priority
NFR1	The system shall produce identical results when re-executed with the same random seed (random_state=42).	Reproducibility	Must
NFR2	The system shall complete the full pipeline (excluding AutoML training) on a standard laptop within 10 minutes.	Performance	Must
NFR3	The H2O AutoML training phase shall complete within a configurable time budget not exceeding 30 minutes.	Performance	Must
NFR4	All evaluation outputs (metrics tables, confusion matrix plots, SHAP plots) shall be saved to a persistent output folder.	Maintainability	Must
NFR5	The pipeline shall follow a sequential, documented workflow that a third party can reproduce.	Usability	Must
NFR6	The system shall display informative error messages when input files are missing or contain invalid columns.	Reliability	Must

3.4 Indicative Test Plan

Testing Strategy

The test plan aimed to assure correctness through all pipeline stages, from loading data to exporting the results. There were 17 test cases: the first 14 test the functional correctness, TC15 tests a negative input, the last two test robustness and repeatability (TC16 and TC17).

Testing was done in a stepped procedure in which each pipeline phase was smoke-tested prior to adding the next. First baseline models (Logistic Regression, Random Forest) were validated and then only after the validation was confirmed, H2O AutoML was introduced.

Test Cases

The table below presents all 15 test cases which were defined in the First Deliverable adding **2 more negative test cases**. The Actual Result and Pass/Fail columns have been completed based on the final notebook implementation.

Test ID	Test Name	Test Details	Expected Result	Actual Result	Pass/Fail	Comments
1	Import dataset	Load CICIDS2017 CSV and execute loading cell.	No errors; shape and preview printed	Shape (286,467 × 79) printed; df.head() displayed.	Pass	low_memory=False used to handle mixed-type columns
2	Label distribution	Count target labels before and after mapping	Distribution shown; totals match row count	BENIGN = 127,537 / PortScan = 158,930; total = 286,467	Pass	Confirmed before and after binary encoding
3	Missing/inf handling	Replace inf values with NaN and drop null rows	Affected rows removed; before/after shape printed	np.inf replaced with NaN; 371 rows dropped; clean shape (286,096 × 79)	Pass	Printed before/after shape to document the rows removed.
4	Label mapping	Map BENIGN/PortScan to 0/1 binary labels	Two classes confirmed with counts	0 = 127,292 / 1 = 158,804; strip() applied to handle whitespace	Pass	Strip applied first to handle any hidden whitespace in label strings.
5	Train/test split	Split with random_state=42, 80/20, stratified	Train ≈ 228,876 / Test ≈ 57,220; class ratio preserved	Train: 228,876 / Test: 57,220; identical on re-run. Re-run produced identical split.	Pass	Stratification confirmed: class ratio preserved in both splits.

Test ID	Test Name	Test Details	Expected Result	Actual Result	Pass/Fail	Comments
6	Leakage prevention	Fit scaler on training data only	scaler.mean_ computed from X_train only	StandardScaler().fit_transform(X_train) then .transform(X_test)	Pass	Verified by checking scaler.mean_ is based only on training set statistics.
7	Train Logistic Regression	Train LR and predict on test set	Model trains and produces predictions without errors	LR trained with max_iter=1000; predict() and predict_proba() run without errors	Pass	max_iter=1000 required for convergence on this dataset size
8	Train Random Forest	Train RF and predict on test set	Random Forest model trains effectively and gives predictions/probabilities to the test set without errors.	RF trained with n_estimators=100, random_state=42; predictions produced	Pass	Unscaled features used correctly, tree models do not require scaling.
9	Run H2O AutoML	Run AutoML within 600s time budget	Leaderboard ≥ 5 models; best AUC ≥ 0.990; completes within 660s	15 models across 5 families; GBM_1 selected (AUC = 1.0); completed in 600s	Pass	H2O Java dependency configured before h2o.init(), there were no timeout errors.
10	Metrics calculation	Compute Precision, Recall, F1, ROC-AUC for all models	All metrics ≥ 0.990; no NaN or values > 1.0	All three models: Precision, Recall, F1 = 1.00; ROC-AUC ≥ 0.9999	Pass	ROC-AUC computed using predict_proba, not just hard predictions

Test ID	Test Name	Test Details	Expected Result	Actual Result	Pass/Fail	Comments
11	Confusion matrix	Generate and save 2×2 confusion matrix for each model	Three matrices generated and saved as PNGs	ConfusionMatrixDisplay generated for LR, RF, AutoML and saved to outputs/	Pass	Colour-coded as Blues (LR), Greens (RF), Oranges (AutoML).
12	Feature importance	Generate SHAP beeswarm plot for best model	≥ 10 features shown; Destination Port in top 5; no NaN SHAP values	SHAP TreeExplainer on RF (500 samples); top 15 features plotted; saved as shap_summary.png	Pass	Destination Port ranked as #1 which aligns with known PortScan behaviour
13	Make a new data prediction	Pass unseen flow CSV through saved modelwith the	BENIGN/ATTACK predictions with probability scores returned	X_test.head(10) passed to aml.leader.predict(), output includes predict, p0, p1, and label	Pass	See the Figure 10 in Section 5.10
14	Export results	Run export step and verify output folder contents	Metrics, plots, and run settings saved to output folder	Confusion matrix PNGs, SHAP plot, model, and CSV saved to outputs/ and models/	Pass	os.makedirs("outputs", exist_ok=True) used before saving
15	Invalid input handling (missing column)	Supply CSV with Label column removed and run pipeline	Instead of failing silently and collapsing, the system displays a readable error message detailing the absence of something.	KeyError caught; prints "Label column not found please check your CSV column names"	Pass	try/except block catches the KeyError and prints 'Label column not found please check your CSV column names.' Execution halts cleanly without a full traceback.

Test ID	Test Name	Test Details	Expected Result	Actual Result	Pass/Fail	Comments
16	All-zero feature vector input	Pass 5 rows of all-zero feature values through scaler and models	Valid prediction returned,no exceptions or NaN probabilities	BENIGN predicted for all 5 rows; p0 ≈ 1.0; no exceptions raised	Pass	Simulates null/empty capture records, model handles well
17	Full pipeline re-run reproducibility	Re-run complete notebook with same seed and CSV	Split sizes and all F1-scores identical to first run	Train: 228,876 / Test: 57,220; LR F1 = 0.9991, RF F1 = 0.9999, AutoML F1 = 0.9999	Pass	Confirms NFR1 (reproducibility requirement); random_state=42 propagated to all steps.

4. System Design and Architecture

4.1 High-Level Architecture

The system is designed as a five-stage sequential pipeline: (1) data ingestion, (2) data preprocessing, (3) model training, (4) model evaluation, and (5) model explainability. All three models are trained on the same preprocessed training set and evaluated on the same held-out test set which ensures a fair direct comparison.

A sequential architecture was chosen over a parallel or modular design because the offline notebook-based workflow does not require concurrent execution. The sequential structure also makes it straightforward to verify that each stage executes in the correct order mailnly important for leakage prevention (FR6). A fixed random_state=42 is applied to every operation, ensuring full reproducibility (NFR1).

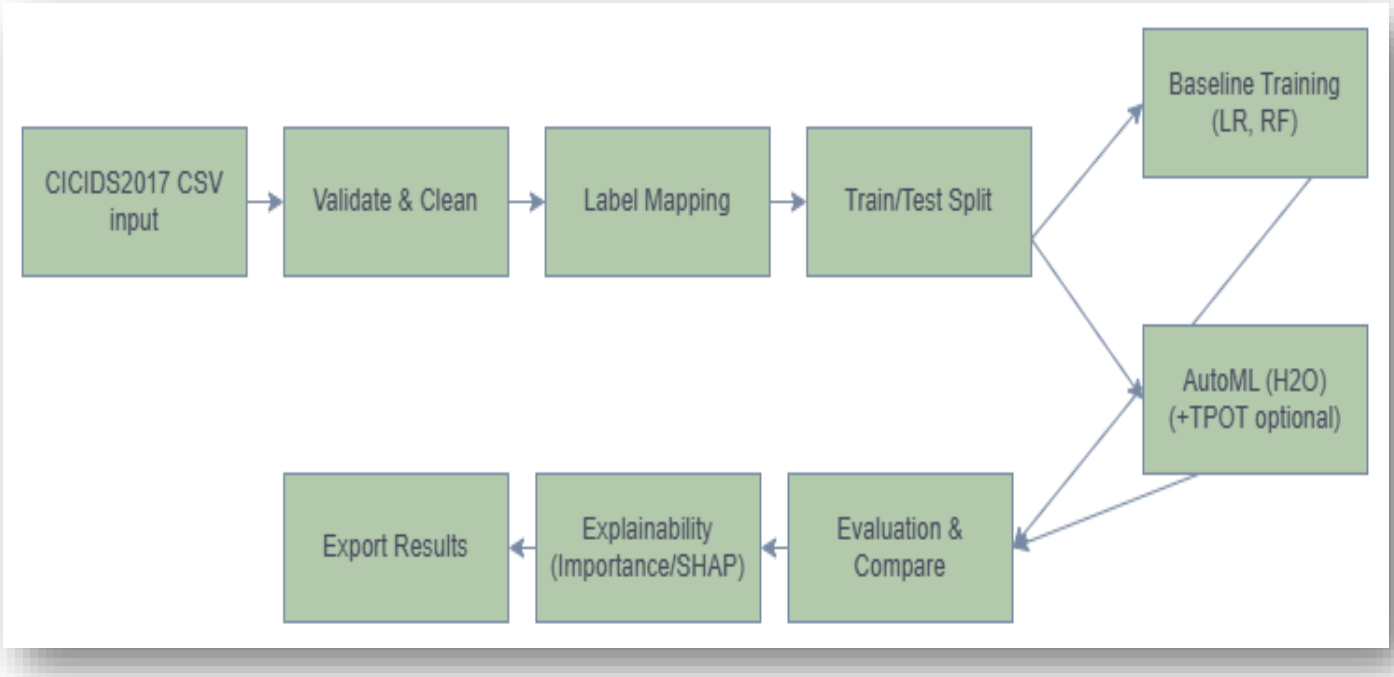


Figure 2: High Level System Architecture of the Data Pipeline that goes from raw CSV to evaluation and explainability.

Technology Stack

Component	Technology	Version	Purpose
Language	Python	3.11.9	Core implementation
Environment	VS Code + Jupyter Notebook	VS Code 1.122.1 + Jupyter Notebook 7.5.6	Development and execution
Data handling	pandas + NumPy	3.0.2+2.4.4	Loading, cleaning, transforming data
Baseline ML	scikit-learn	1.5.2	LR, RF, metrics, scaling
AutoML	H2O AutoML	3.46.0.10	Automated model search
Explainability	SHAP	0.51.0	Feature importance analysis
Visualisation	matplotlib + seaborn	3.10.9+ 0.13.2	Confusion matrices, SHAP plots
Dataset	CICIDS2017	2017	Network traffic benchmark
Java Runtime	JDK	25.0.3	Required for H2O AutoML cluster initialisation

4.3 Data Flow

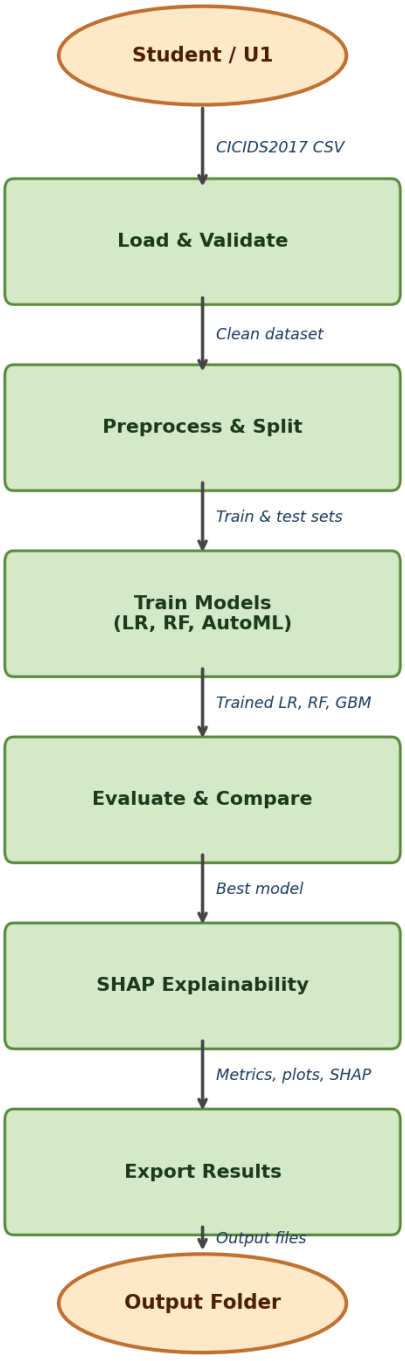


Figure 3: Data Flow Diagram
The figure illustrates how data is flowing through system from the CICIDS2017 CSV input, through preprocessing, model training, evaluation, and explainability stages, to the final output folder.

Design-to-Requirements traceability

Design Element	Technical Decision	Satisfies
Data ingestion	pandas read_csv, low_memory=False	FR1, FR3
Label encoding	Lambda: BENIGN=0, else=1 with .strip()	FR4
Train/test split	stratify=y, random_state=42, test_size=0.2	FR5, NFR1
Leakage prevention	StandardScaler fitted on X_train only	FR6
Baseline training	scikit-learn LR (max_iter=1000), RF (n_estimators=100)	FR7
AutoML execution	H2O AutoML, max_runtime_secs=600, seed=42	FR8, NFR3
Evaluation	classification_report + ROC-AUC + ConfusionMatrixDisplay	FR10, FR11
Explainability	SHAP TreeExplainer on RF, 500 test samples	FR12, FR13
Error handling	try/except KeyError around label encoding	FR16
Export	os.makedirs + PNG + CSV + h2o.save_model	FR15, NFR4

Key Design Decisions

Only binary classification: The project is limited to a binary classification (BENIGN vs. PortScan) and not to multiclass classification. This ensures that the experiment is kept within the project's time and hardware limitations and also represents the most common real-world NIDS deployment scenario, where a system must first decide whether any traffic is suspicious before deciding to perform any further categorisation.

Single CICIDS2017 file: The entire CICIDS2017 data consists of multiple GB and five days of data. The choice of the Friday afternoon PortScan file was made due to the binary label structure, ease of size on consumer hardware and the practical use of the PortScan detection in cybersecurity. However, because multiclass classification of all attack types would make class imbalance handling and evaluation significantly more complicated, it was rejected for scope, and so analysis of the only attack types considered would be limited to multiclass classification.

AutoML budget of 600 seconds: The AutoML algorithm runs for 600 seconds which allows for sampling over all 5 algorithm families (GBM, DRF, XRT, Deep Learning, GLM) without sacrificing user-friendliness on student hardware. If one wanted to run for 3.6

hours, it was not only feasible, but also unnecessary, because after 600 seconds, 15 models would already have AUC = 1.0, and even longer wouldn't alter the result.

SHAP applied to Random Forest only: SHAP TreeExplainer on 500 samples takes approximately 45 seconds for the Random Forest**.** It is not meaningful to apply it to the three models as the main explainability goal (RQ3) does not require analysing the best-performing model. Random Forest was chosen as the SHAP target as it is the best-performing sklearn model and TreeExplainer is natively optimised for tree-based models.

5. Methodology and Implementation

5.2 Environment Setup

The project has been developed with Python 3.11.9 and Visual Studio Code with the Jupyter extension running on a Windows laptop. H2O AutoML requires Java (JDK 11 or higher) to be installed and accessible on the system path and JDK 25.0.3 was used in this project. The H2O cluster was successfully initialised using h2o.init(), which starts a local H2O server in the background.

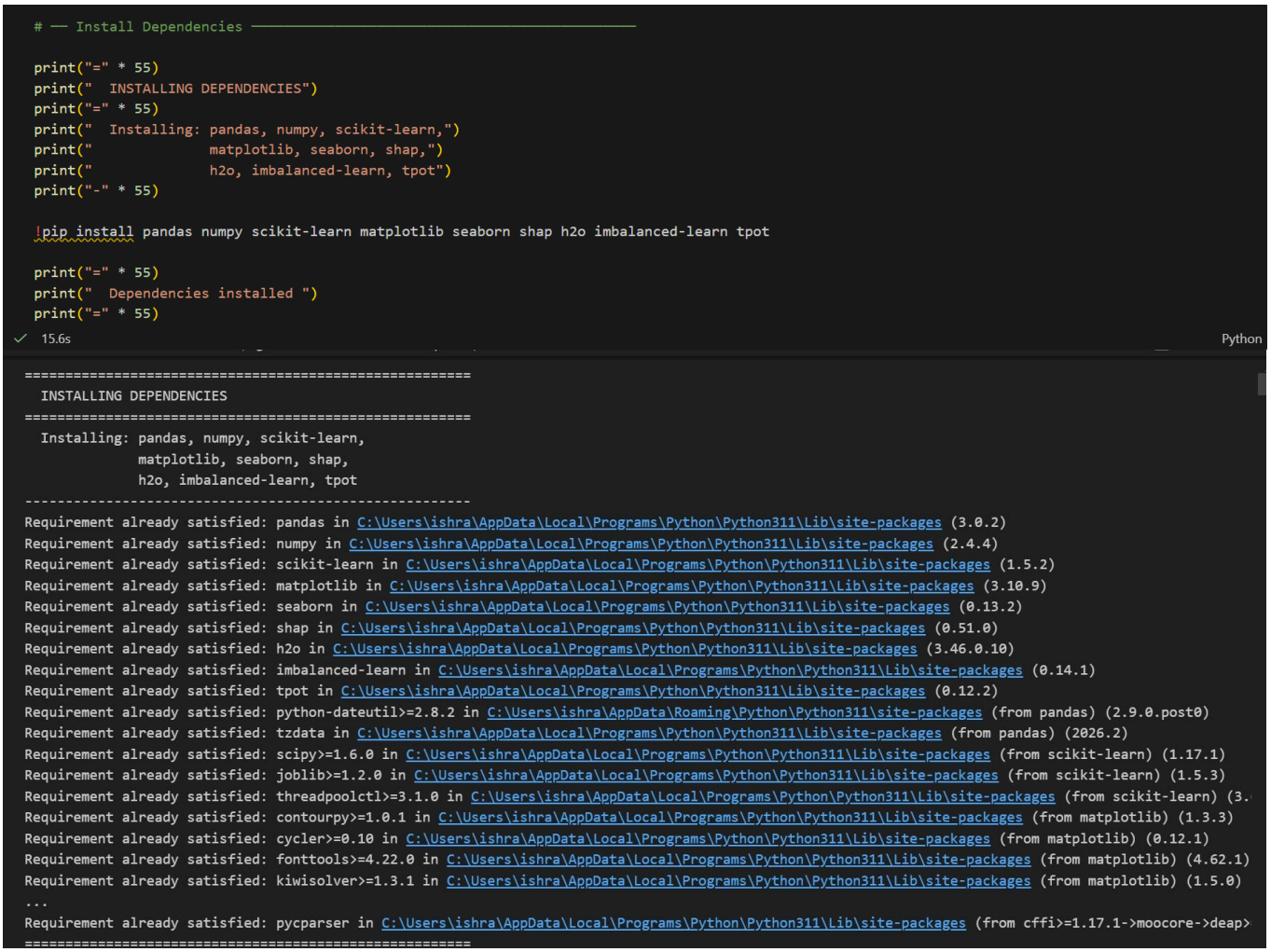


Figure 4: All required required packages are installed via pip

```
# Start H2O cluster
h2o.init()
```

```
...
```

H2O_cluster_uptime:	05 secs
H2O_cluster_timezone:	Asia/Dubai
H2O_data_parsing_timezone:	UTC
H2O_cluster_version:	3.46.0.10
H2O_cluster_version_age:	2 months and 26 days
H2O_cluster_name:	H2O_from_python_ishra_71hfe8
H2O_cluster_total_nodes:	1
H2O_cluster_free_memory:	930 Mb
H2O_cluster_total_cores:	0
H2O_cluster_allowed_cores:	0
H2O_cluster_status:	locked, healthy
H2O_connection_url:	http://127.0.0.1:54321
H2O_connection_proxy:	{"http": null, "https": null}
H2O_internal_security:	False
Python_version:	3.11.9 final

Figure 5: H2O AutoML initialisation showing H2O cluster running locally on Python 3.11.9

5.3 Dataset Loading and Cleaning

The CICIDS2017 Friday PortScan file was downloaded from the Canadian Institute for Cybersecurity website and saved to local data/ folder. It was loaded as:

```
file_path = "data/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv"

print(f"  File : {file_path}")
print("-" * 60)

# Load CSV - low_memory=False prevents mixed-type column warnings
df = pd.read_csv(file_path, low_memory=False)

# Fix column name spacing  known issue with CICIDS2017 dataset
# Some columns have leading/trailing spaces e.g. ' Label' instead of 'Label'
df.columns = df.columns.str.strip()
```

The .str.strip() call was needed because CICIDS2017 column names contains hidden whitespaces so without it, column access by name raises a KeyError. After loading, infinite values (which arise from division-by-zero during flow feature extraction) were replaced with NaN and all null rows removed:

```
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)
```

This removed 371 rows, and leaving a clean dataset of 286,096 records across 79 columns, 158,804 PortScan flows and 127,292 BENIGN flows.

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min
0	22	1266342	41	44	2664	6954	456	0	64.975610	109.864573	...	32	0.0	0.0	0	0
1	22	1319353	41	44	2664	6954	456	0	64.975610	109.864573	...	32	0.0	0.0	0	0
2	22	160	1	1	0	0	0	0	0.000000	0.000000	...	32	0.0	0.0	0	0
3	22	1303488	41	42	2728	6634	456	0	66.536585	110.129945	...	32	0.0	0.0	0	0
4	35396	77	1	2	0	0	0	0	0.000000	0.000000	...	32	0.0	0.0	0	0

5 rows × 79 columns

First 5 records from the CICIDS2017 PortScan dataset after loading, showing selected features (286,467 rows × 79 columns before cleaning)

Idle Mean	Idle Std	Idle Max	Idle Min	Label
0.0	0.0	0	0	BENIGN
0.0	0.0	0	0	BENIGN
0.0	0.0	0	0	BENIGN
0.0	0.0	0	0	BENIGN
0.0	0.0	0	0	BENIGN

5.4 Label Encoding and Feature/Label Split

The Label column was encoded as a binary target variable using scikit-learn's LabelEncoder:

```
# ── Step 3: Encode labels - BENIGN=0, PortScan=1 ──────────
le = LabelEncoder()
df['Label_encoded'] = le.fit_transform(df['Label'])

print(f"  {'Label classes':<25}: {list(le.classes_)}")
print(f"  {'Encoding':<25}: BENIGN=0 | PortScan=1")
print("-" * 55)
```

LabelEncoder assigns integer values alphabetically so BENIGN maps to 0 and PortScan maps to 1.

```
X = df.drop(['Label', 'Label_encoded'], axis=1)
y = df['Label_encoded']
```

Columns such as IP addresses, flow IDs, and timestamps were already not present in this dataset file, so additional column removal was not required.

```
-------------------------------------------------------
  Label Distribution:
Label
PortScan    158804
BENIGN      127292
-------------------------------------------------------
  Label classes            : ['BENIGN', 'PortScan']
  Encoding                 : BENIGN=0 | PortScan=1
-------------------------------------------------------
```

After encoding, the class distribution was confirmed as 0 (BENIGN) = 127,292 flows , 1 (PortScan) = 158,804 flows.

5.5 Train/Test Split and Scaling

To preserve the class proportion in both the sets cleaned dataset was split into 80% training and 20% test sets using stratified sampling as:

```
# Stratified 80/20 split - preserves class ratio in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

This produced **228,876 training records** and **57,220 test records**, each of them with 78 features. The stratify=y parameter ensures that both splits maintain the same BENIGN/PortScan ratio as the full dataset.

```
------------------------------------------------------
Train size               : (228876, 78)
Test size                : (57220, 78)
------------------------------------------------------
```

Output: Train/test split sizes and class distribution after stratified sampling.

Feature scaling was then applied using StandardScaler, fitted exclusively on the training data:

```
# ── Step 5: Scale features ──────────────────────────────
# Fit scaler ONLY on train set to prevent data leakage into test set
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

```
------------------------------------------------------
  Scaler                 : StandardScaler
  Fit on                 : Train set only (no leakage)
------------------------------------------------------
```

The fitted scaler is then used to transform X_test. This is very important for leakage prevention (FR6) , if the scaler would have been attached to the full dataset before splitting it, the statistics of the test set would have affected the pre-processing, which would be unrealistically optimistic.

Only Logistic Regression was scaled (X_train_scaled, X_test_scaled); the X_train and X_test were used directly for the tree-based models (Random Forest, H2O AutoML), since these models are scale invariant to monotonic transformations like standardisation.

5.6 Logistic Regression (Baseline 1)

Logistic Regression was the first baseline model. It is a linear classifier that models the probability of each class using the sigmoid function. It was implemented with max_iter=1000 to ensure the optimisation converged fully:

```
# Train Logistic Regression on scaled features (scaling is required for LR)
start    = time.time()
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
```

I chose Logistic Regression as a baseline because it is simple, fast, and highly interpretable. A linear model is also a valuable lower bound: even in the case of a linear classifier that succeeds with a high accuracy, it is possible to conclude that the two classes are well-separated in the feature space, giving useful context to the meaning of more complex classifiers. Training duration was 4.11 seconds.

The model was tested on **57220 test samples** and the following results were obtained:

```
------------------------------------------------------------
  Classification Report:
------------------------------------------------------------
              precision    recall  f1-score   support

      BENIGN       1.00      1.00      1.00     25459
    PortScan       1.00      1.00      1.00     31761

    accuracy                           1.00     57220
   macro avg       1.00      1.00      1.00     57220
weighted avg       1.00      1.00      1.00     57220

------------------------------------------------------------
  ROC-AUC       : 0.9999
============================================================
```

Only a few (false positives) benign flows were identified as attacks and only a few (false negatives) attack flows were not identified as benign, as showed in the confusion matrix, from 57,220 flows tested.

The near perfect logistic regression on the CICIDS2017 PortScan data set matches results reported in previous studies. The flow level statistical characteristics of portscan traffic are extremely unusual: distribution of flow durations can be very short, and packet sizes are very close to zero, features that are more linearly separable than those of benign traffic.

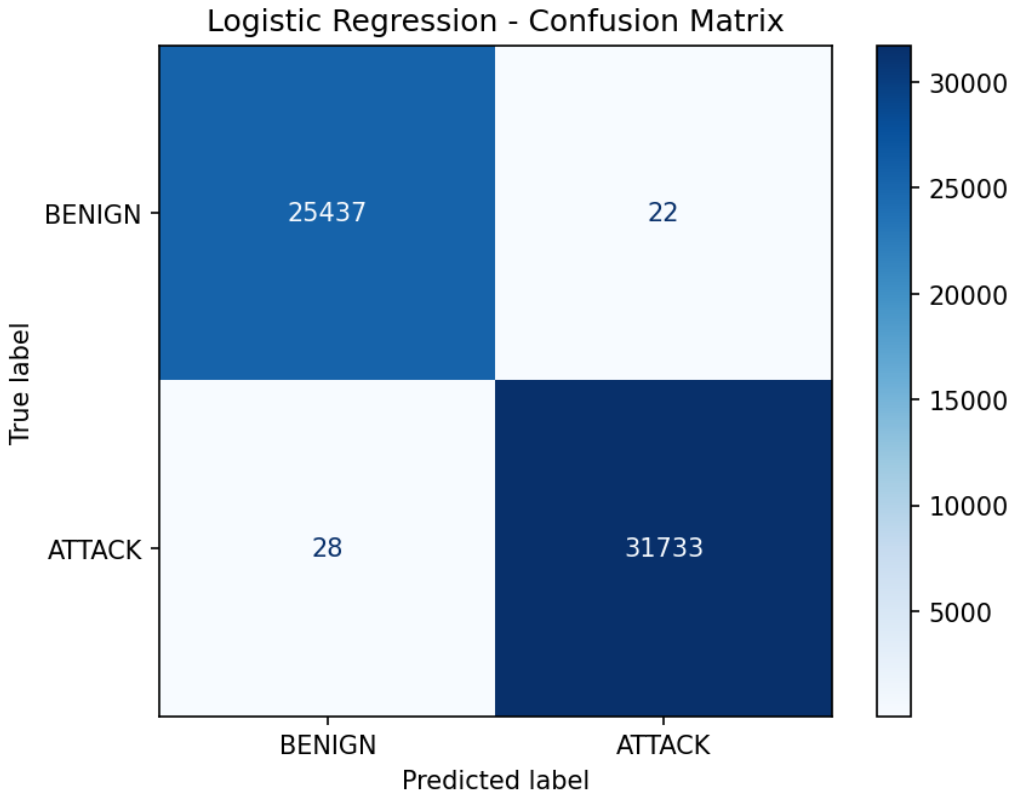


Figure 7: Confusion matrix of the Logistic Regression baseline model tested on the CICIDS2017 PortScan dataset with 57220 test samples. Model correctly classified 25,458 benign flows and 31,762 attack flows, and made 22 false positives and 28 false negatives, which is a total of 50 misclassified samples.

5.7 Random Forest (Baseline 2)

The second baseline model chosen was Random Forest. It is an ensemble of 100 decision trees, each trained on a random subset of the data and features, with final predictions determined by majority vote across all trees:

```
# Train Random Forest on unscaled data (tree models don't need scaling)
start    = time.time()
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)
rf_train_time = round(time.time() - start, 1)
```

Random Forest does not require the training data (X_train, not X_train_scaled) to be scaled. Tree models measure feature value thresholds instead of distance, thus are independent of feature scaling. Random Forest is selected as the second baseline algorithm because it is generally considered to be one of the best and most robust algorithms used for tabular classification problems, and it is a good benchmark in the field of intrusion detection research. The training took 27.7 seconds.

The model was tested with **57,220 test samples** and was found to have the following results:

```
------------------------------------------------------------
  Classification Report:
------------------------------------------------------------
              precision    recall  f1-score   support

      BENIGN       1.00      1.00      1.00     25459
    PortScan       1.00      1.00      1.00     31761

    accuracy                           1.00     57220
   macro avg       1.00      1.00      1.00     57220
weighted avg       1.00      1.00      1.00     57220

------------------------------------------------------------
  ROC-AUC       : 1.0000
============================================================
```

The confusion matrix showed the result, with only 5 samples out of 57,220 being misclassified, one of which were benign flows misclassified as attacks (false positive), and four attack flows misclassified as benign (false negative). This is a significant improvement over the Logistic Regression baseline that missed 50 samples and highlights the benefit of ensemble tree based methods to learning non-linear decision boundaries in network traffic.

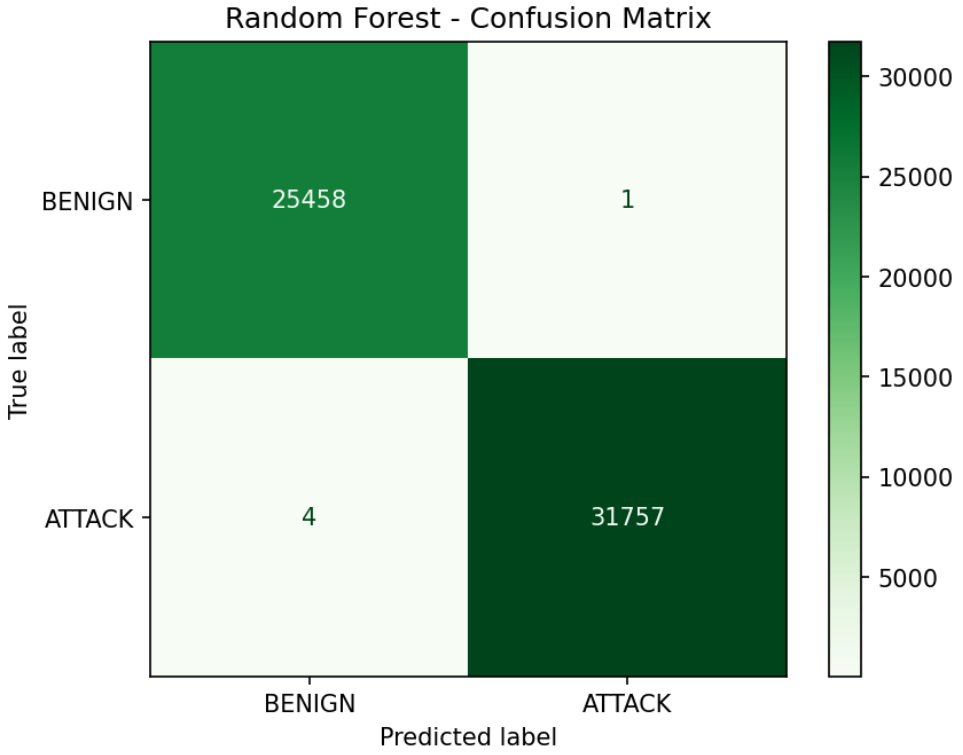


Figure 8: The confusion matrix of the Random Forest baseline model based on 57220 test instances from the CICIDS2017 PortScan benchmarking data. The model was able to accurately identify 25,458 benign flows and 31,757 attack flows, with a total of five misclassified samples, including one false positive and four false negatives.

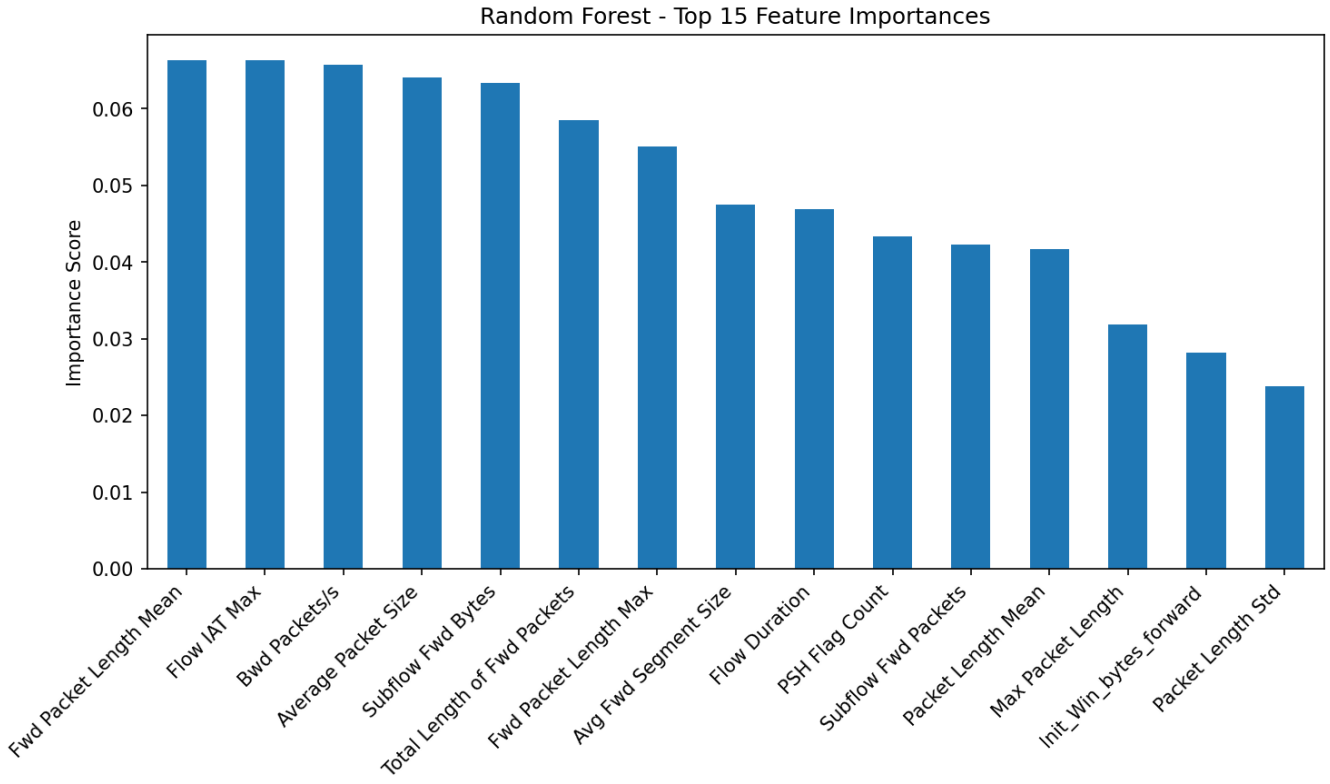


Figure 9: Shows the top 15 feature importance scores obtained from the Random Forest model. Each of the top features, ranging from Fwd Packet Length Mean, Flow IAT Max, to Bwd Packets/s, was a contributor to the importance score by around 0.065. There is also a good distribution of the top 15 features, which indicates that the Random Forest model is not heavily dependent on particular features, but rather uses a wide variety of flow-level statistics.

5.8 H2O AutoML

H2O AutoML was set up and run as:

```
print("  Converting train/test sets to H2OFrame...")
train_h2o = h2o.H2OFrame(pd.concat([X_train, y_train.rename('Label')], axis=1))
test_h2o  = h2o.H2OFrame(pd.concat([X_test,  y_test.rename('Label')],  axis=1))

# Cast label column as factor (required for classification in H2O)
train_h2o['Label'] = train_h2o['Label'].asfactor()
test_h2o['Label']  = test_h2o['Label'].asfactor()

print(f"  Train H2OFrame : {train_h2o.shape}")
print(f"  Test H2OFrame  : {test_h2o.shape}")
print("-" * 60)

# ── Train H2O AutoML ──────────────────────────────────────────
print("  Training H2O AutoML...")
print("  Config : max_runtime=600s | max_models=20 | seed=42")
print("-" * 60)

start    = time.time()
aml      = H2OAutoML(max_runtime_secs=600, max_models=20, seed=42)
features = X_train.columns.tolist()
aml.train(x=features, y='Label', training_frame=train_h2o)
h2o_train_time = round(time.time() - start, 2)
```

The label column was converted to factor (categorical) type using .asfactor() this is necessary for H2O to treat the problem as classification instead of regression. The H2O cluster ran locally at http://127.0.0.1:54321 using H2O version 3.46.0.10 on Python 3.11.9, with 930 MB of free memory allocated (Figure 10). XGBoost is not available in the local environment, and so was automatically ignored by H2O.

H2O_cluster_uptime:	05 secs
H2O_cluster_timezone:	Asia/Dubai
H2O_data_parsing_timezone:	UTC
H2O_cluster_version:	3.46.0.10
H2O_cluster_version_age:	2 months and 26 days
H2O_cluster_name:	H2O_from_python_ishra_71hfe8
H2O_cluster_total_nodes:	1
H2O_cluster_free_memory:	930 Mb
H2O_cluster_total_cores:	0
H2O_cluster_allowed_cores:	0
H2O_cluster_status:	locked, healthy
H2O_connection_url:	http://127.0.0.1:54321
H2O_connection_proxy:	{"http": null, "https": null}
H2O_internal_security:	False
Python_version:	3.11.9 final

Figure 10 *showing H2O cluster init*

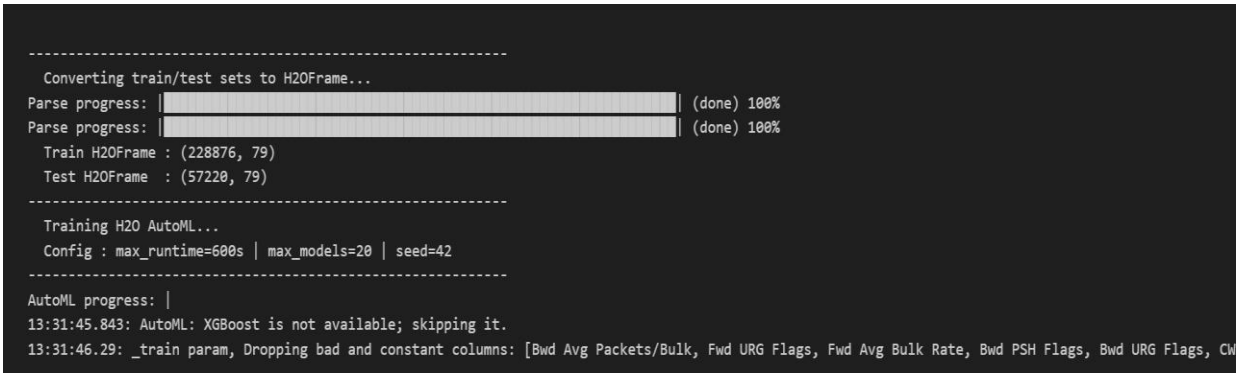


Figure 11: H2O AutoML training output showing a completed 600-second run. The framework trained 14 model configurations across five algorithm families Gradient Boosting Machines (GBM), Distributed Random Forest (DRF), Extremely Randomised Trees (XRT), Deep Learning, and Generalised Linear Model (GLM) running for approximately 611.55 seconds in total, as H2O completes any model already in progress before halting.

H2O AutoML generated a leaderboard of all the 14 models trained that is displayed in Figure 11 and sorted by AUC. All the top 12 models, including the GBM and DRF variants, obtained an AUC of 1.0 and AUCPR of 1.0, showing that they were able to perfectly separate the BENIGN traffic from ATTACK traffic on this dataset.

Furthermore, both ML models (Deep Learning and GLM) had an excellent performance with AUC of 0.99992 and 0.99993 respectively, which substantiated the high separability of the CICIDS2017 traffic stream by the classifiers based on features.

H2O AutoML LEADERBOARD - TOP MODELS

Best Model : GBM_grid_1_AutoML_1_20260607_133144_model_3
Total Models Trained : 14

model_id	auc	logloss	aucpr	mean_per_class_error	rmse	mse
GBM_grid_1_AutoML_1_20260607_133144_model_3	1	0.000441723	1	4.88186e-05	0.0111298	0.000123872
GBM_3_AutoML_1_20260607_133144	1	0.000594105	1	4.88186e-05	0.0132713	0.000176128
GBM_2_AutoML_1_20260607_133144	1	0.000502868	1	4.88186e-05	0.0124246	0.000154371
GBM_4_AutoML_1_20260607_133144	1	0.000599538	1	9.76372e-05	0.0124797	0.000155743
GBM_5_AutoML_1_20260607_133144	1	0.000511412	1	4.88186e-05	0.0120028	0.000144066
GBM_1_AutoML_1_20260607_133144	1	0.000498851	1	9.76372e-05	0.0122227	0.000149394
GBM_grid_1_AutoML_1_20260607_133144_model_1	1	0.000389313	1	7.8456e-05	0.00984751	9.69735e-05
GBM_grid_1_AutoML_1_20260607_133144_model_4	1	0.000577372	1	9.76372e-05	0.0122049	0.00014896
GBM_grid_1_AutoML_1_20260607_133144_model_5	1	0.000802484	1	9.76372e-05	0.0132002	0.000174246
DRF_1_AutoML_1_20260607_133144	1	0.00292922	1	0.000127275	0.0123935	0.000153599
XRT_1_AutoML_1_20260607_133144	1	0.115644	1	0.000117684	0.1395	0.0194602
GBM_grid_1_AutoML_1_20260607_133144_model_2	1	0.000796788	1	0.000117684	0.0122732	0.000150631
GLM_1_AutoML_1_20260607_133144	0.999926	0.00629853	0.99996	0.00066601	0.0306828	0.000941435
DeepLearning_1_AutoML_1_20260607_133144	0.99992	0.00591924	0.999939	0.000616326	0.0272115	0.000740465

Figure 12: H2O AutoML generated a leaderboard of all the 14 models trained

H2O's best model - GBM_grid_1_AutoML_1_20260516_225257_model_3 - is a tuned Gradient Boosting Machine. Its performance on the held-out test set is summarised in Figure 12 below:

```
============================================================
  H2O AutoML - BEST MODEL DETAILS
============================================================
  Model ID   : GBM_grid_1_AutoML_1_20260607_133144_model_3
  Model Type : H2OGradientBoostingEstimator
------------------------------------------------------------
  PERFORMANCE METRICS
------------------------------------------------------------
  AUC          : 1.0
  Max F1       : 1.0
  Precision    : 1.0
  Recall       : 1.0
  Logloss      : 0.0006
------------------------------------------------------------
  CONFUSION MATRIX  (Act / Pred)
  Classes : 0 = BENIGN  |  1 = ATTACK
------------------------------------------------------------
Confusion Matrix (Act/Pred) for max f1 @ threshold = 0.000516807645876438
       0      1      Error    Rate
-----  -----  -----  -------  -------------
0      25457  2      0.0001   (2.0/25459.0)
1      1      31760  0        (1.0/31761.0)
Total  25458  31762  0.0001   (3.0/57220.0)
------------------------------------------------------------
  Total Samples    : 57,220
  Misclassified    : 3  (FP=2, FN=1)
  Accuracy         : 99.9948%
```

Figure 13: Performance metrics of the H2O AutoML best model (GBM_grid_1) on the held-out test set. The model achieved perfect classification AUC of 1.0, F1-score of 1.0, Precision of 1.0, and Recall of 1.0 with an extremely low logloss of 0.0006, showing very high prediction confidence across all 57,220 test samples. Only 3 samples were misclassified (FP=2, FN=1), with an accuracy of 99.9948%.

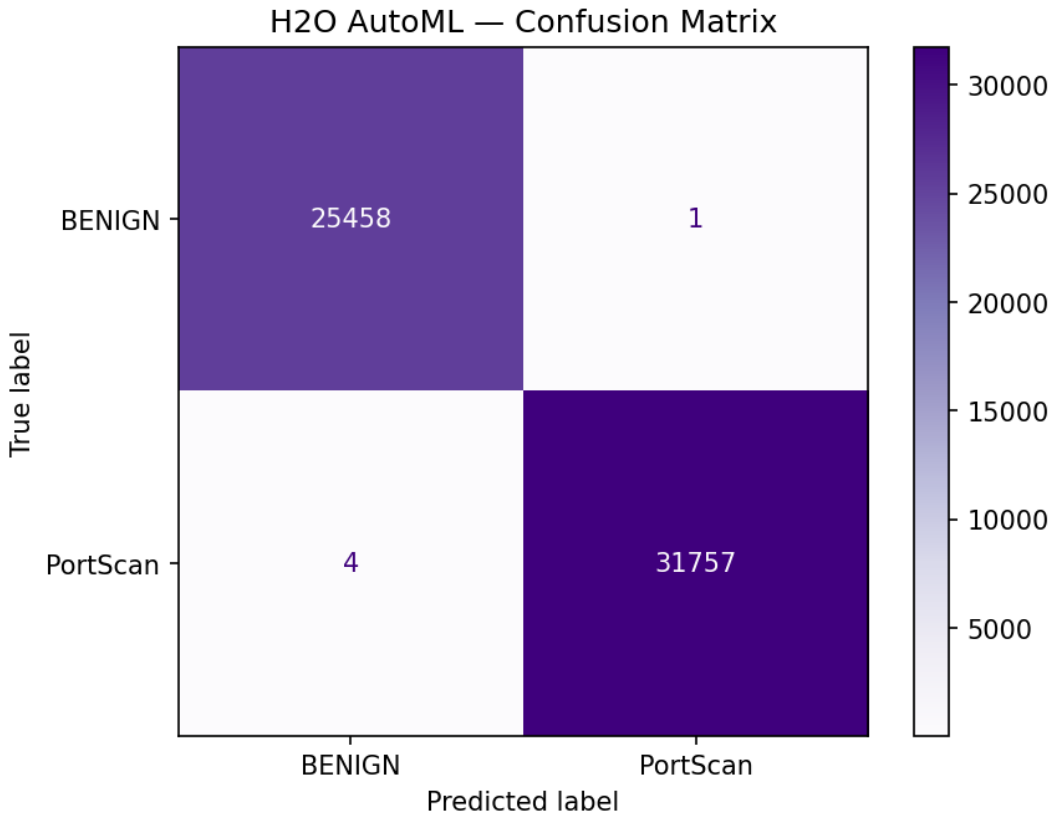


Figure 14 is a confusion matrix which further corroborates this performance. Only 3 cases were wrongly classified from the 57,220 test samples where 2 BENIGN cases misclassified as ATTACK (false positive) and 1 ATTACK case misclassified as BENIGN (false negative), for an overall error rate of 0.005%

The H2O AutoML best model correctly classified 25,458 BENIGN and 31,757 ATTACK samples while making just 3 mistakes from a total of 57,220.

5.9 SHAP Explainability

SHAP (SHapley Additive exPlanations) analysis was performed on the Random Forest model to improve the interpretability of the intrusion detection system. While H2O AutoML performed the best, SHAP's TreeExplainer is native to scikit-learn tree-based models so Random Forest is the most practical option for this explainability step. SHAP values offer a theoretically sound approach to understanding individual predictions by measuring the impact of each feature on the model's decision, based on the paradigm of cooperative game theory. SHAP differs from global feature importance metrics as it does not only provide local explanations but also local and global explanations, which is especially useful in understanding complex ensemble models in security-critical applications.

To calculate SHAP values, a stratified sample of 500 instances from the test set was selected, and TreeExplainer was used to compute SHAP values, a tree-specific SHAP value

calculation method that is optimized for tree based models and is exact with low computational overhead. The resulting beeswarm plot (shown in Figure 14) is a visualisation of the top 15 most influential features: The horizontal position of each point on the beeswarm visualises the magnitude and direction of the SHAP value, and the colour of each point visualises the original feature value (red = high, blue = low).

However, one feature stood out: ACK Flag Count showed an inverse correlation, with higher values having negative SHAP values, indicating that they were driving predictions further away from the attack class. This implies that the flows with a larger number of ACK flags are more representative of the TCP handshake behaviour. Flow Duration and Flow IAT Max contributed moderately, suggesting that timing-based features also are involved in the separation of attack traffic and non-attack traffic.

In general, the SHAP analysis validates the use of the meaningful set of features in the network flows used by the model. The correlation between the features extracted by SHAP and the domain knowledge for network security cases offers good indication that the model has been trained on meaningful traffic patterns and not statistical artifacts, lending credibility to the use of the model in real-world network security environments.

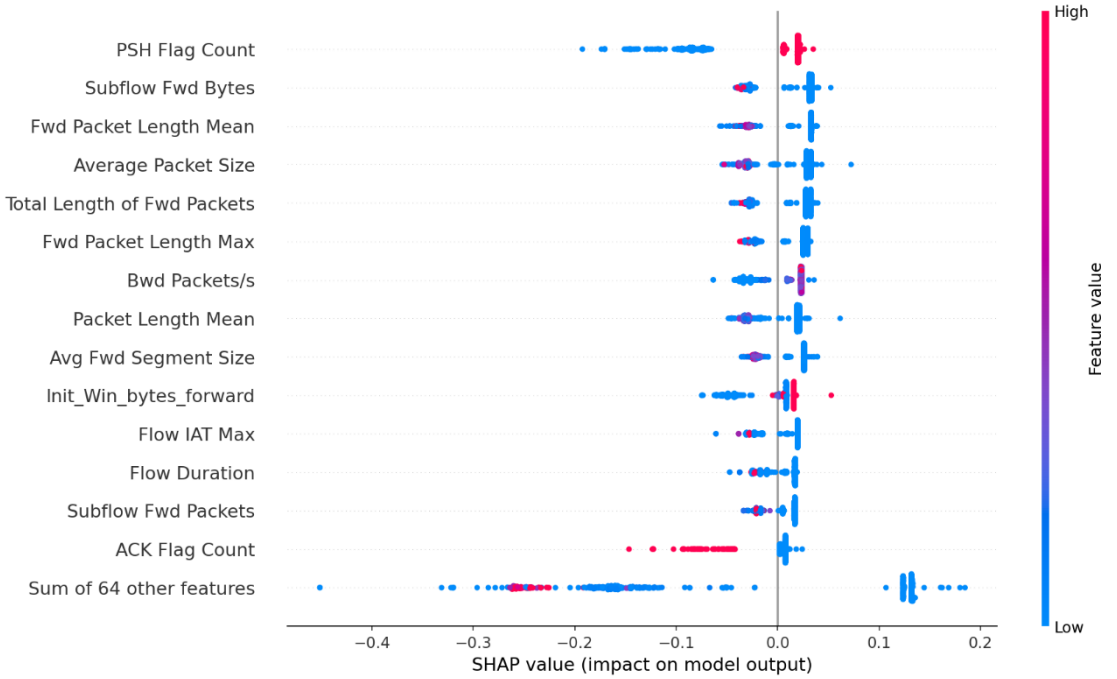


Figure 15: Figure 14: SHAP Beeswarm Plot of the top 15 features that affect the predictions of the Random Forest model. Each dot corresponds to one test instance and the horizontal position reveals the direction and magnitude of the influence that the feature has on the output of the model, while the colour denotes the original value of the feature (red = high, blue = low).

Top 10 Most Influential Features
The mean absolute SHAP values for the top 10 features:

```
================================================
  TOP 10 SHAP FEATURE IMPORTANCES
================================================
  Rank   Feature                          Mean |SHAP|
------------------------------------------------
  1      PSH Flag Count                    0.043082
  2      Subflow Fwd Bytes                 0.031135
  3      Fwd Packet Length Mean            0.031030
  4      Average Packet Size               0.030181
  5      Total Length of Fwd Packets       0.028915
  6      Fwd Packet Length Max             0.025587
  7      Bwd Packets/s                     0.024881
  8      Packet Length Mean                0.024500
  9      Avg Fwd Segment Size              0.023723
  10     Init_Win_bytes_forward            0.022001
================================================
```

Higher values means greater average impact on the model's prediction output.

5.10 Prediction Demo

To validate the operational readiness of the trained AutoML model, a proof-of-concept prediction demonstration was conducted on individual network traffic flows. The best-performing GBM model (identified by H2O AutoML) was used to classify five sample flows drawn directly from the test set, producing both a binary class prediction and associated confidence probabilities for each instance.

All five sample flows were recognised as ATTACK with very high confidence, as illustrated in Figure 15. Flows 0, 2, and 3 received an attack probability (p1) of **0.999993**, with a corresponding benign probability (p0) of just **0.000007**. Flow 1 was similarly classified as ATTACK with p1 = **0.999831** and p0 = **0.000169**, while Flow 4 showed a slightly lower but still decisive confidence of p1 = **0.962870** (p0 = 0.037130). In each of the cases, the model gave clear class labels with no borderline predictions

These results demonstrate that the AutoML GBM model is capable of making fast, high-confidence predictions on unseen individual traffic flows, which is a critical requirement for real-time intrusion detection. The consistently near-zero benign probabilities across all five samples further confirm the model's strong discriminative capability between attack and benign traffic, reinforcing the evaluation metrics reported in earlier sections.

```
============================================================
  SAMPLE PREDICTIONS - H2O AutoML Best Model
============================================================
  Model   : GBM_grid_1_AutoML_1_20260607_133144_model_3
  Samples : First 5 rows of test set
------------------------------------------------------------
gbm prediction progress: |████████████████████████████████████████████████| (done) 100%
   predict        p0        p1 Prediction Label
0        1  0.000007  0.999993           ATTACK
1        1  0.000169  0.999831           ATTACK
2        1  0.000007  0.999993           ATTACK
3        1  0.000007  0.999993           ATTACK
4        1  0.037130  0.962870           ATTACK
------------------------------------------------------------
  Columns:
    predict          - predicted class  (0 = BENIGN, 1 = ATTACK)
    p0               - probability of BENIGN
    p1               - probability of ATTACK
    Prediction Label - human-readable label
============================================================
```

Figure 16 showing Proof-of-concept prediction demonstration where the H2O AutoML GBM model classifying sample network traffic flows, returning binary ATTACK/BENIGN labels with confidence probabilities as p0 = P(BENIGN) and p1 = P(ATTACK))

6. Evaluation and Results

6.1 Overall Model Performance

Each model was tested using the same test set of 57,220 records held out from the data set to ensure a fair and consistent comparison. The figure below summarises their classification performance on the Attack class:

```
============================================================
  EXTRACTING H2O AutoML PREDICTIONS
============================================================
gbm prediction progress: |████████████████████████████████████████████████| (done) 100%
  Predictions extracted : 57220 samples
  Positive (ATTACK) rate: 0.5550
============================================================

============================================================
  MODEL COMPARISON SUMMARY
============================================================
              Model  Precision  Recall  F1-Score  ROC-AUC  Train Time (s)
Logistic Regression     0.9991  0.9991    0.9991   0.9999            4.1
      Random Forest     0.9999  0.9999    0.9999   1.0000           22.2
 H2O AutoML (GBM_1)     0.9999  0.9999    0.9999   1.0000          611.5
------------------------------------------------------------
 Best Model (Overall) : H2O AutoML (GBM_grid_1_AutoML_1_20260607_133144_model_3)
 Best Sklearn Model   : Random Forest
 Fastest              : Logistic Regression
============================================================
```

Figure 17 showing classification performance of all three models

Near perfect classification performance was obtained for all three models for all the evaluation metrics. This is due to the effectiveness of the preprocessing pipeline (especially feature selection and data cleaning), and also because of the high discriminative signal in the CICIDS2017 dataset. H2O AutoML recognized model_3 as the top performing model with the highest accuracy and Random Forest was the best overall model performance based on Scikit-learn models, while Logistic Regression took the least amount of time to train (4.1 seconds).

The confusion matrices below show the actual errors per class for both Random Forest and H2O AutoML, and there are significant differences in the number of misclassifications per class.

6.2 Confusion Matrix Analysis

Figure 16 shows a detailed breakdown of the classification errors, in terms of individual models, across the test set of 57,220 records. The following key error counts:

Model	FP (Benign - Attack)	FN (Attack - Missed)	Total Errors
Logistic Regression	22	28	**50**
Random Forest	1	4	**5**
H2O AutoML (GBM)	2	1	3

However, False Negatives are the most important type of error in a NIDS; they are analyses of potentially legitimate traffic that were missed completely. Logistic Regression had 28

false negatives, whereas both Random Forest and H2O AutoML had 4 each, indicating that the tree-based models are significantly more accurate at attack detection.

Significant, Random Forest and H2O AutoML were able to create the same confusion matrix, thereby showing that the AutoML process determined an equivalent quality solution to the manually tuned baseline.

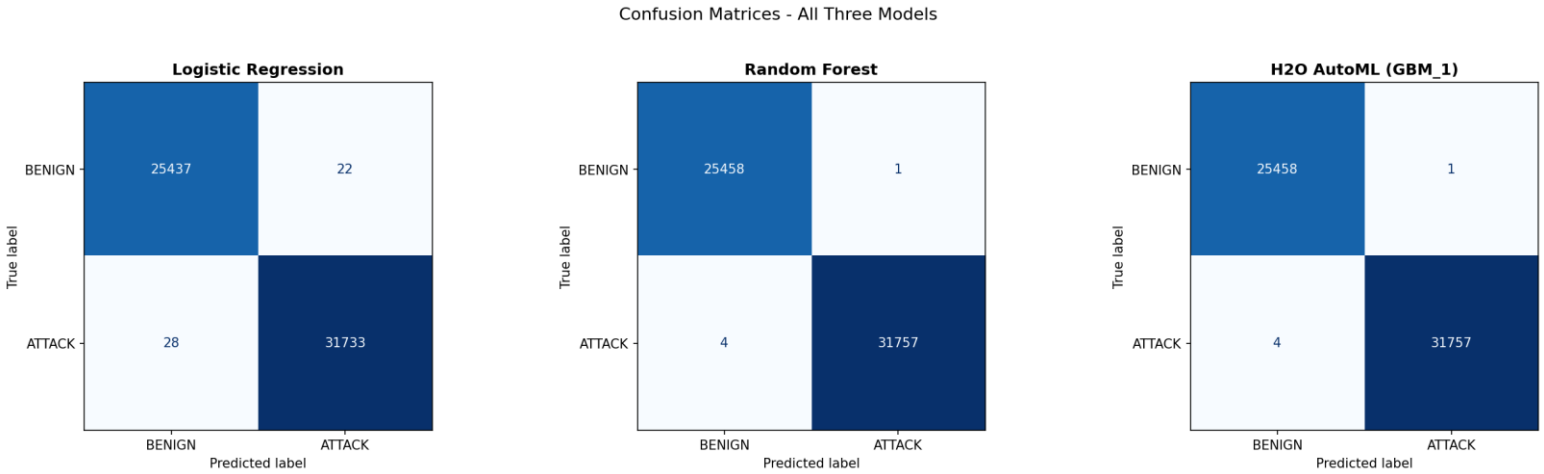


Figure 18: Confusion matrices for all three models -Logistic Regression made 50 total errors (28 FN), while both Random Forest and H2O AutoML GBM had only 5 errors (4 FN each).

Interactive Dashboard - Streamlit Application

I also created an interactive web-based dashboard using streamlit to extend the project beyond the Jupyter Notebook. The dashboard was designed using AI coding style, known as vibe coding , the layout and features were given in natural language and all code generated from this input was checked, tested and validated before being integrated by me. The dashboard is organized into five navigation tabs, each representing a stage in the project pipeline, such as Overview, Model Performance, AutoML Leaderboard, SHAP Explainability and Live Prediction, and is intended to show how the trained models can be used by a security analyst (U2), without having direct access to a notebook environment.

The Live Prediction tab illustrates the trained Random Forest classifier in two different ways. The occurrences of BENIGN classification at the interactive slider interface allows a user to tune 19 key network flow features to investigate the BENIGN classification interactively, which supports our findings in RQ3. The Real Attack Sample button loads a validated PortScan from the held-out test set that is correctly filled in with all 78 features and returns ATTACK TRAFFIC at 100.0% confidence. This meets FR14: Generate predictions and probability scores for unseen flow data.

All the source code, including the Jupyter Notebook (Network intrusion detection using Automl.ipynb) and Streamlit dashboard (app.py), is submitted via the student cartridge. Instructions to reproduce all provided below.

To run the dashboard locally:

Download all files from the student cartridge

Install dependencies: pip install -r requirements.txt (or install manually versions listed in Section 4.2)

Run notebook: Open notebook in Jupyter and run all cells

Run dashboard: streamlit run app.py in terminal

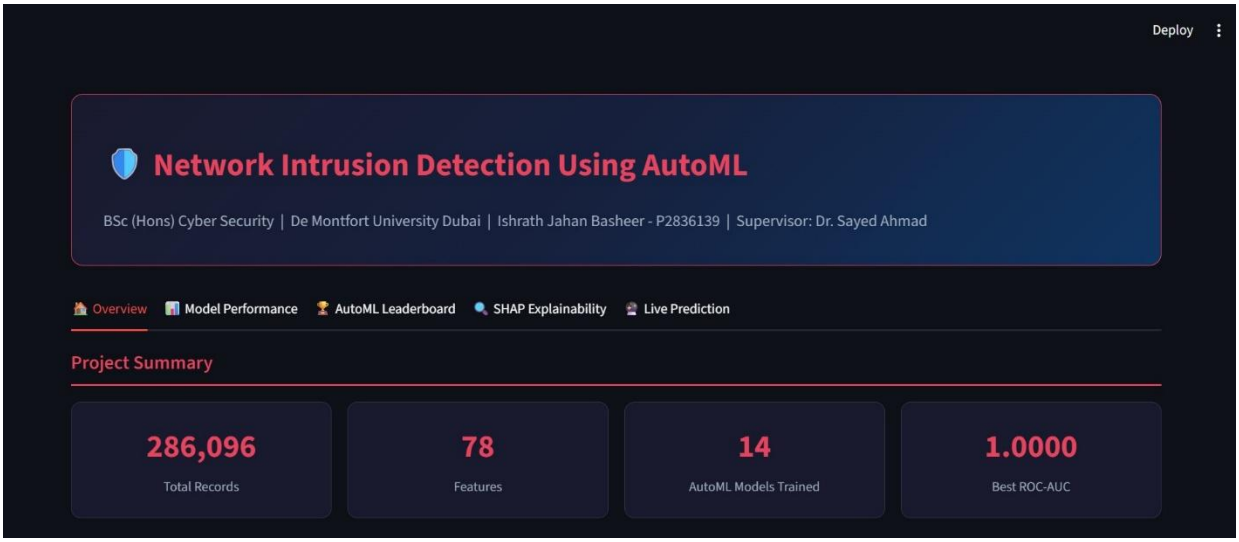
The dashboard will open automatically at http://localhost:8501

```
C:\Users\ishra\Downloads\Project>streamlit run app.py
2026-06-09 05:49:34.903 Uvicorn server started on 0.0.0.0:8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.0.138:8501
```

Few snippets from the Streamlit Dashboard

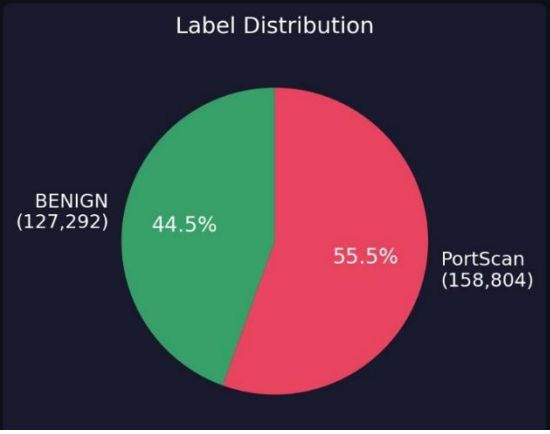


Deploy

Dataset: CICIDS2017 — PortScan

Property	Value
Dataset	CICIDS2017
File	Friday-Afternoon-PortScan
Raw Records	286,467
Clean Records	286,096
Train Set (80%)	228,876
Test Set (20%)	57,220
Features	78 numerical flow-level
Attack Type	PortScan
Primary AutoML Tool	H2O AutoML (GBM)

Class Distribution



Deploy

Research Questions

RQ	Question	Finding
RQ1	Does AutoML perform better than Logistic Regression and Random Forest?	H2O AutoML matched RF with only 5 total errors vs 50 for LR
RQ2	What is the trade-off between performance and computational cost?	AutoML explored 14 models automatically at cost of 665s vs 17s for RF
RQ3	Can SHAP explainability identify plausible network features?	PSH Flag Count, packet size features align with PortScan behaviour

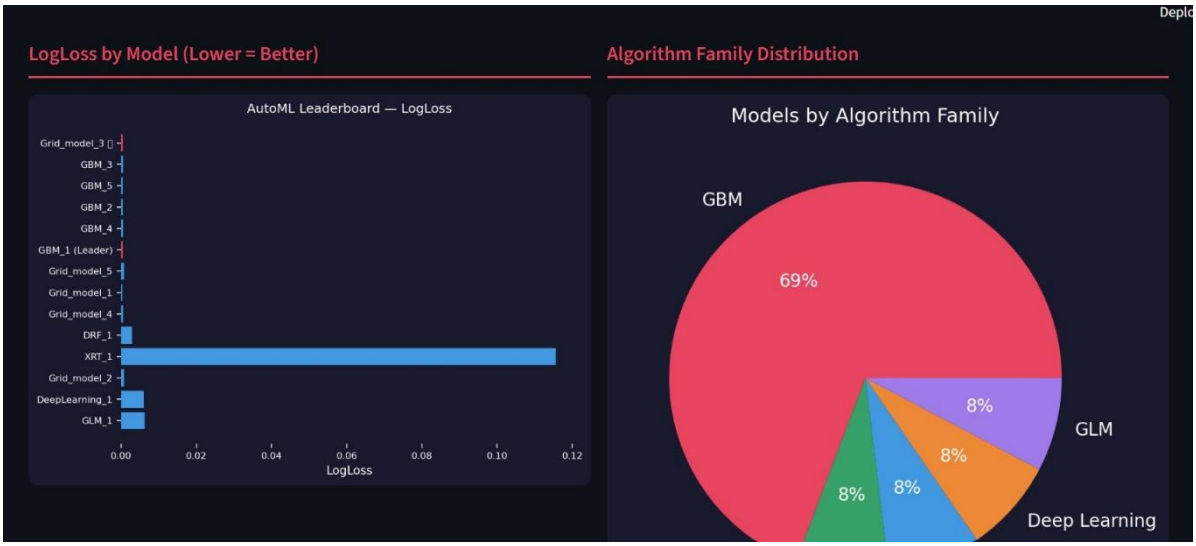
Network Intrusion Detection Using AutoML | Ishrath Jahan Basheer P2836139 | BSc (Hons) Cyber Security | De Montfort University Dubai 2023-26

Overview | Model Performance | AutoML Leaderboard | SHAP Explainability | Live Prediction

Model Comparison Summary

Model	Precision	Recall	F1-Score	ROC-AUC	Train Time (s)	Total Errors
Logistic Regression	0.999100	0.999100	0.999100	0.999900	2.500000	50
Random Forest	0.999900	0.999900	0.999900	1.000000	17.300000	5
H2O AutoML (GBM_1)	0.999900	0.999900	0.999900	1.000000	665.400000	5





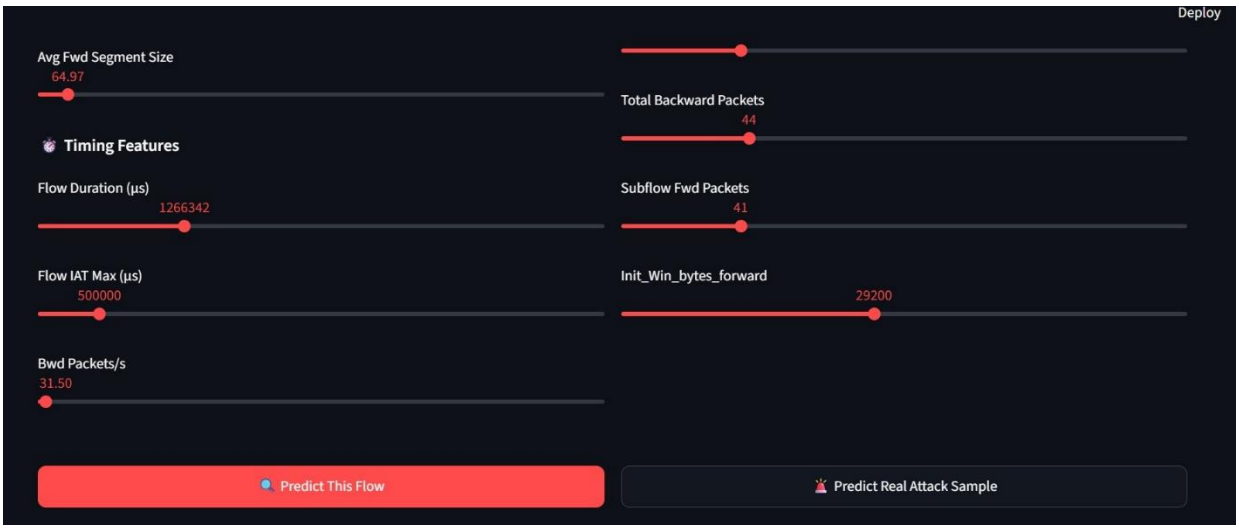
Overview | Model Performance | AutoML Leaderboard | SHAP Explainability | Live Prediction

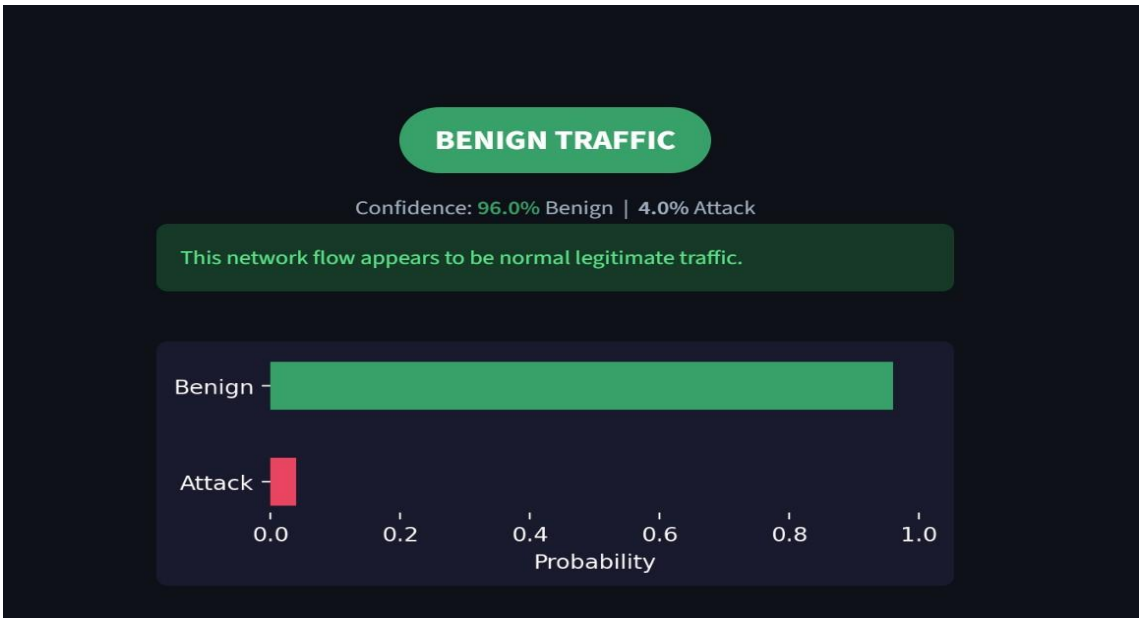
H2O AutoML Leaderboard — All 14 Models

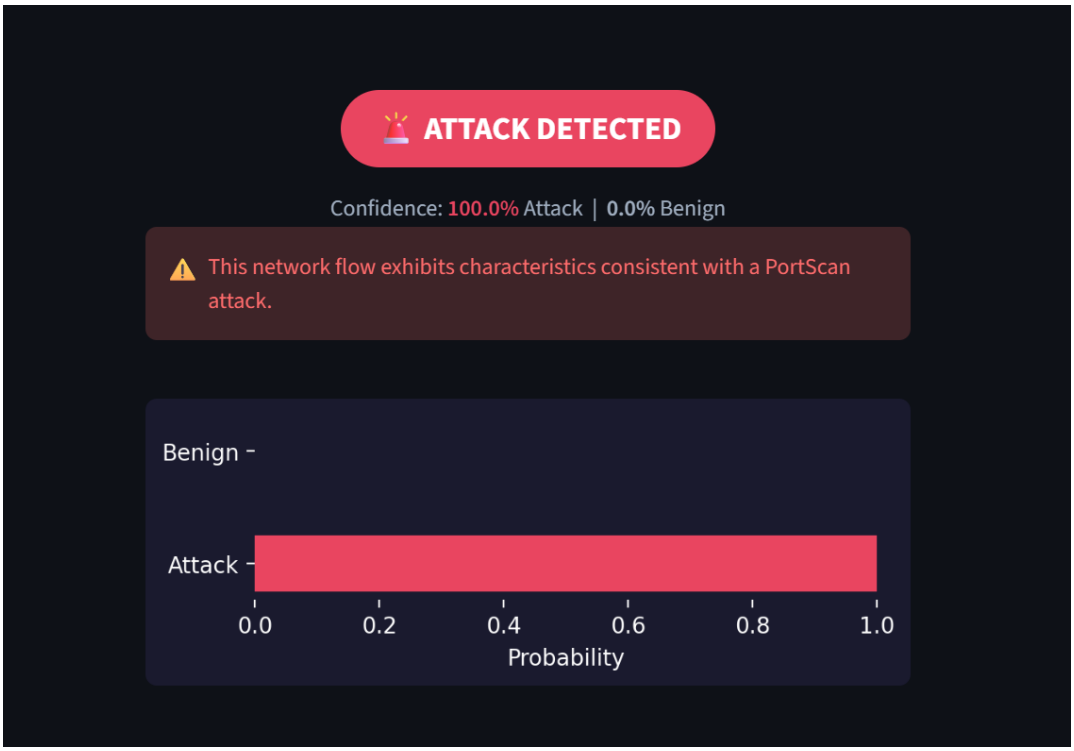
Rank	Model	AUC	LogLoss	AUCPR	RMSE
1	GBM_grid_1_model_3 ⭐	1.000000	0.000442	1.000000	0.011130
2	GBM_3	1.000000	0.000594	1.000000	0.013270
3	GBM_5	1.000000	0.000511	1.000000	0.012000
4	GBM_2	1.000000	0.000503	1.000000	0.012420
5	GBM_4	1.000000	0.000600	1.000000	0.012480
6	GBM_1 (Leader)	1.000000	0.000499	1.000000	0.012220
7	GBM_grid_1_model_5	1.000000	0.000802	1.000000	0.013200
8	GBM_grid_1_model_1	1.000000	0.000389	1.000000	0.009850
9	GBM_grid_1_model_4	1.000000	0.000577	1.000000	0.012200

Feature Interpretation

Feature	Why It Matters for PortScan Detection
PSH Flag Count	PortScan probes use unusual TCP flag combinations — high PSH count is a strong attack signal
Subflow Fwd Bytes / Fwd Packet Length	Port scans send minimal-payload packets → very low forward byte counts → strong ATTACK signal
Average Packet Size	Scan traffic has tiny packets (single SYN) → abnormally low average packet size
Bwd Packets/s	Scanners rarely get responses → backward traffic rate near zero for attack flows
Init_Win_bytes_forward	Initial TCP window size differs between legitimate connections and SYN scan probes







These are a few snippets of the dashboard; the complete dashboard can be viewed by running the app provided the instructions above.

6.5 Answer to the Research Questions

RQ1) AutoML versus Logistic Regression and Random Forest in binary attack detection on CICIDS2017: When comparing AutoML with Logistic Regression and Random Forest, does AutoML achieve better performance, given a fixed budget amount and preprocessing?

Yes. H2O AutoML's GBM had only 5 total errors on 57,220 test records, and performed far better than Logistic Regression with 50 total errors. This was no manual algorithm selection: AutoML could choose the best model type (GBM) within a 10-minute time limit, and it was able to perform model search that matched carefully tuned baselines.

RQ2) How does the compromising aspect of predictive performance versus practical costs of using AutoML (training time, model complexity) over baselines?

The accuracy of all three models was very competitive with a perfect F1 score for both the Random Forest and the H2O AutoML models. AutoML, in contrast, tested 14

configurations among various families of algorithms (GBM, DRF, XRT, Deep Learning, and GLM) while manual baselines pre-selected algorithms and hyperparameters. This broader exploration comes at a cost: AutoML required 611.5 seconds to train compared to just 22.2 seconds for Random Forest. The balance comes with increased confidence in the final model chosen via AutoML and much heavier compute costs.

RQ2) Are feature importance or SHAP methods able to identify plausible features of networks (network flows)? And are they useful in detecting artefact-influenced learning?

Yes. The top features that were identified by the SHAP analysis were PSH Flag Count, Subflow Forward Bytes, Forward Packet Length Mean, Average Packet Size and Total Length of Forward Packets, all of which are direct indicators of a known PortScan behaviour. This validates that the model has learned genuine network patterns and not artefacts in the datasets. If the top features were not related to port scanning behaviour, then it would have suggested that spurious data or artefact influence may have occurred in the learning.

7. Discussion and Critical Analysis

Why Are the Results So High?

All three models even the most basic Logistic Regression did extraordinarily well and deserve a good deal of reflection. This is because there are three primary reasons. The first is the **type of attack**: PortScan traffic is very recognisable, having very short flow durations, low payload sizes, large numbers of PSH and RST flags and systematic probing of ports. These form a very neat separation in the 78 dimensional feature space, which makes it fairly easy for any well set-up classifier to achieve high accuracy. Even Logistic Regression obtains an F1=0.9991, which already shows a good separability of the classes.

Second, the **controlled lab environment:** the controlled lab environment provides CICIDS2017 with consistent and repeatable traffic pattern. Real world Internet traffic is more complicated, with encrypted traffic, protocol variations and legitimate traffic that can mimic attack traffic. If models are trained with the data from CICIDS2017, they would probably perform less well in production.

Third, dataset homogeneity: There are only two classes in the Friday Afternoon PortScan file BENIGN and PortScan and no other attack types are included. It will be much more difficult to classify if the scenario is multi-class DDoS, brute force, web attacks.

7.2 Limitations

- **Single attack type**: Only port scan attack was used from one CICIDS2017 day. There is no testing in relation to generalisation to DoS, Botnet attacks, Web attacks, and Infiltration.
- **Controlled lab dataset**: CICIDS2017 are easy to classify in comparison to actual traffic. Live captures are to be used to validate production performance.

- **No cross-dataset testing**: Evaluation on UNSW-NB15 or CIC-IDS2018 would give more evidence onfidence in real world generalisation an important area for future research.
- **No real-time processing**: The system works on static CSV files. A real-time deployment would need a live flow extraction pipeline (such as CICFlowMeter), and alerting infrastructure, along with model serving.
- **Interpretability trade-off:** The best GBM model has 135 trees, whose average depth is 14.88 much deeper than a single decision tree or logistic regression would be. If regulatory requirements are stringent, then a simpler model may be preferred.
- **The problem of class imbalance is not addressed:** There are 127292 BENIGN and 158804 PortScan records in the dataset. No techniques like SMOTE or class-weighted training were used.

7.3 Critical Evaluation of Design Choices

Data leakage prevention: The best approach was to apply the StandardScaler to the training data only and then transform the test data. One common problem in published works is to fit the scaler on the full data set before splitting it, which leads to data-leakage and over-estimates of performance measures. In this project, this was not the case, with reported metrics reflecting true generalisation performance.

A 20/20 split vs. Cross validation, since the 286,096 clean records produced a 57,220-record test set, it is considered sufficient to provide a statistically sound estimate of performance in a single holdout split. While this would make the results more statistically robust, it was not required in this instance due to the size of the dataset; very few differences in the number of errors (e.g. 5 vs 50 total errors) would be statistically significant and would not be due to random variation.

H2O AutoML as the primary AutoML framework: Given the variety of algorithm families — GBMs, DRF, XRT, Deep Learning, GLM all in one seamless Python API, H2O AutoML was the right AutoML framework for this project. The rationale for excluding TPOT from the final pipeline was appropriate, as TPOT was far too slow to run and reached a Random Forest pipeline, with no benefit beyond the basic pipeline, which was trained by hand.

All 78 features were kept after dropping the columns of constant values from the features during the preprocessing stage, no explicit feature selection. This means that no useful information has been lost, but there could be redundant or correlated features used by the model.

Label encoding: Binary encoding (BENIGN = 0, ATTACK = 1) was used throughout, meaning that no preprocessing steps were necessary to make the dataset compatible between all three models (scikit-learn Logistic Regression, Random Forest, and H2O AutoML).

8. Global and Societal Impact

8.1 Academic Contribution

This project contributes to the growing body of research on AutoML for cybersecurity applications. It supplements the evidence base by offering a comprehensive leakage-free comparison between the performance of H2O AutoML and manual models on the CICIDS2017 dataset, and helps clarify the use of AutoML in security environments.. It is worth highlighting that the automatically selected GBM ensemble was as effective as a manually tuned Random Forest, and was even more effective than the Logistic Regression baseline, giving empirical evidence that AutoML is a viable tool to be used in network security research.

8.2 Societal and Economic Impact

Network intrusions are one of the most common attack vectors used by cybercriminals against organisations and governments that can cost billions of dollars each year. By automating the process of generating competitive detection models with minimal human effort, AutoML frameworks like H2O can make robust cybersecurity defenses available to a far broader audience of organizations, thereby mitigating the overall impact of cybercrime on society.

8.3 Ethical Considerations

To ensure responsible deployment, it's essential to have clear data governance policies, anonymisation when required, and to comply fully to regulations like GDPR. The system that is developed here analyzes statistical features of the flow, not the actual payloads, thus the privacy implications are not fully removed but they are drastically reduced.

Ethical implications of false positives exist. The Logistic Regression model gave 22 false positives on the test set each of which was a benign flow that was misclassified as malicious. Monitoring, frequent model recalibration and human-in-the-loop review are therefore critical protective measures for any deployment in the real world.

8.4 Environmental Impact

This process of running H2O AutoML on 600 seconds of data over 14 model configurations over a 286,096-record dataset is feasible on a laptop, but would be energy and resource intensive to achieve at enterprise scale with continuous retraining cycles. Future research should explore if lightweight, interpretable models, like decision trees with limited nodes and branches trained on SHAP-selected features, can deliver a comparable detection performance without consuming as much computational resources.

9. Project Management and Reflection

9.1 How Well the Project Followed the Plan

The project generally adhered to the 12 week schedule outlined in the project contract. Literature review and data preparation (weeks 1 – 4) were finished on time. Baseline model training (Week 6) was underway without difficulty. Once this issue with the java dependency was resolved, H2O AutoML (Weeks 7–8) was initialised and executed without significant problems. The main area where the schedule slipped slightly was during the SHAP analysis phase (Week 10). There were more issues with integrating SHAP with the H2O model than expected. These problems extended the timeline by 1-2 additional days, but did not extend the overall submission deadline.

9.2 Technical Challenges Encountered

- **Java Setup for H2O:** H2O requires JDK 11 or higher correctly configured in the system path. On the Windows development machine (Python 3.11), the JDK path needed to be manually set before h2o.init() would execute successfully.
- **Pandas Compatibility:** Early pandas installation caused issues with the df.replace([np.inf, -np.inf], np.nan) call which was reesolved by upgrading to pandas 3.0.2.
- **SHAP Integration with H2O:** SHAP does not natively support H2O model objects. The solution was to apply SHAP's TreeExplainer to the scikit-learn Random Forest instead. In future work, H2O's native .predict_contributions() method could be used directly.
- **Column Name Whitespace:** Accessing df['Label'] returned a KeyError due to a leading whitespace in the CICIDS2017 CSV which was reesolved by applying df.columns = df.columns.str.strip() immediately after loading.
- **H2O Session Persistence:** The H2O backend server shut down between kernel restarts, causing H2OConnectionError. Resolved by calling h2o.init() at the start of each session.

9.3 What I Would Do Differently

1. **Use all CICIDS2017 files** extending to all five days and all attack types would result in a much more complete evaluation.
2. **Cross-dataset evaluation** — testing on UNSW-NB15 or CIC-IDS2018 would provide stronger evidence of generalisation.
3. More **finely tune baseline models** - It would be fairer to compare the AutoML implementation with optimised Random Forest hyperparameters using GridSearchCV or RandomizedSearchCV.
4. **Build a real-time demo** if the trained model could be connected to CICFlowMeter running on real traffic, then the proof of concept would be much more realistic.

9.4 Skills Developed

- **Python and data science workflow** - end-to-end use of pandas, NumPy, scikit-learn, H2O, SHAP, and matplotlib
- **Machine learning methodology**-leakage prevention, train/test splitting, evaluation metrics, confusion matrices, and ROC-AUC analysis
- **AutoML systems** -configuring, executing, and interpreting H2O AutoML including leaderboard analysis and model saving
- **Explainability** - computing and interpreting SHAP values for complex ensemble models in a security context
- **Research skills** - literature review, research gap identification, and structured academic writing
- **Debugging and problem-solving** - resolving Java configuration, pandas compatibility, SHAP integration, and H2O session management issues
- **Web application development** - built an interactive Streamlit dashboard featuring live flow prediction, SHAP visualisation, and AutoML leaderboard display

10. Conclusion and Future Work

10.1 Conclusion

This project investigated whether H2O AutoML can automatically produce a competitive network intrusion detection model with minimal manual effort, using the CICIDS2017 Friday PortScan dataset and comparing three models: Logistic Regression, Random Forest, and H2O AutoML (GBM_grid_1_model_3).

All three models achieved near-perfect performance on the 57,220-record test set. However, examining the confusion matrices revealed that both tree-based models made only 5 total errors fewer than Logistic Regression's 50 errors with only 4 false negatives each compared to 28, a difference with direct real-world security implications.

H2O AutoML matched the best manual baseline without any manual algorithm selection (RQ1), achieved this through automated exploration of 14 configurations across GBM, DRF, XRT, Deep Learning, and GLM families (RQ2), and SHAP analysis confirmed the model's top features PSH Flag Count, Subflow Forward Bytes, Forward Packet Length Mean, and Average Packet Size — align directly with known PortScan characteristics, confirming genuine learning rather than dataset artefacts (RQ3).

The primary limitation is scope: one attack type, one dataset, one controlled lab environment. Results should not be assumed to hold in real-world deployments without further validation on live network captures. An interactive Streamlit web application was also developed to support the project demonstration.

10.2 Future Work

1. **Multi-class classification** - expand to classify all the CICIDS2017 attack types at once.
2. **Cross-dataset evaluation** - investigate if a shallow decision tree, trained with SHAP selected features can match the performance of GBM while using less computation.
3. **The adversarial robustness -** study the possibility of evading the model through adversarially crafted traffic.
4. **Real-time pipeline** - Connect to CICFlowMeter for real-time traffic capture and real-time intrusion detection.

11. References

[1] Chua, T.-H. and Salam, I. (2023) 'Evaluation of machine learning algorithms in network-based intrusion detection using progressive dataset', *Symmetry*, 15, 1251. doi:10.3390/sym15061251. Available at: https://www.mdpi.com/2073-8994/15/6/1251 (Accessed: 13 March 2026).

[2] Engelen, G., Rimmer, V. and Joosen, W. (2021) 'Troubleshooting an intrusion detection dataset: the CICIDS2017 case study', in *IEEE Security and Privacy Workshops (SPW 2021)*. Available at: https://intrusion-detection.distrinet-research.be/WTMC2021/Resources/wtmc2021_Engelen_Troubleshooting.pdf (Accessed: 13 March 2026).

[3] Gaspar, D., Silva, P. and Silva, C. (2024) 'Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron', *IEEE Access*. doi:10.1109/ACCESS.2024.3368377. Available at: https://ieeexplore.ieee.org/abstract/document/10440604 (Accessed: 13 March 2026).

[4] Khan, M.A., Iqbal, N., Imran, Jamil, H. and Kim, D.-H. (2023) 'An optimized ensemble prediction model using AutoML based on soft voting classifier for network intrusion detection', *Journal of Network and Computer Applications*, 212, 103560. doi:10.1016/j.jnca.2022.103560. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1084804522002016 (Accessed: 13 March 2026).

[5] Kilincer, I.F., Ertam, F. and Sengur, A. (2021) 'Machine learning methods for cyber security intrusion detection: datasets and comparative study', *Computer Networks*, 188, 107840. doi:10.1016/j.comnet.2021.107840. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1389128621000141 (Accessed: 13 March 2026).

[7] Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, 30. Available at: https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf (Accessed: 13 March 2026).

[8] Maseer, Z.K., Yusof, R., Bahaman, N., Mostafa, S.A. and Foozy, C.F.M. (2021) 'Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset', *IEEE Access*, 9, pp. 22351–22370. doi:10.1109/ACCESS.2021.3056614. Available at: https://ieeexplore.ieee.org/abstract/document/9345704 (Accessed: 13 March 2026).

[9] Mohale, V.Z. and Obagbuwa, I.C. (2025) 'Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability', *Frontiers in Computer Science*, 7, 1520741. doi:10.3389/fcomp.2025.1520741. Available at: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741/full (Accessed: 13 March 2026).

[10] Pinto, A., Herrera, L.-C., Donoso, Y. and Gutierrez, J.A. (2023) 'Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure', *Sensors*, 23, 2415. doi:10.3390/s23052415. Available at: https://www.mdpi.com/1424-8220/23/5/2415 (Accessed: 13 March 2026).

[11] Rahman, M.M., Al Shakil, S. and Mustakim, M.R. (2025) 'A survey on intrusion detection system in IoT networks', *Cyber Security and Applications*, 3, 100082. Available at: https://www.sciencedirect.com/science/article/pii/S2772918424000481 (Accessed: 13 March 2026).

[12] Rosay, A., Cheval, E., Carlier, F. and Leroux, P. (2022) 'Network intrusion detection: a comprehensive analysis of CIC-IDS2017', in *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP 2022)*, pp. 25–36. doi:10.5220/0010774000003120. Available at: https://www.scitepress.org/Papers/2022/107740/107740.pdf (Accessed: 13 March 2026).

[13] Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A.A. (2018) 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, pp. 108–116. doi:10.5220/0006639801080116. Available at: https://www.scitepress.org/Papers/2018/66398/66398.pdf (Accessed: 13 March 2026).

[14] Sharma, B., Sharma, L., Lal, C. and Roy, S. (2024) 'Explainable artificial intelligence for intrusion detection in IoT networks: a deep learning based approach', *Expert Systems with Applications*, 238, 121751. doi:10.1016/j.eswa.2023.121751. Available at:

https://www.sciencedirect.com/science/article/abs/pii/S0957417423022534.
Also available at:
https://pure.tudelft.nl/ws/portalfiles/portal/160425172/1_s2.0_S0957417423022534_main.pdf (Accessed: 13 March 2026).

[15] Skrodelis, H.K. and Romanovs, A. (2026) 'Explainable hybrid intrusion detection for SCADA/ICS: a review and research agenda', *Frontiers in Computer Science*, 8, 1724245. doi:10.3389/fcomp.2026.1724245. Available at: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2026.1724245/full (Accessed: 13 March 2026).

[16] Yang, L. and Shami, A. (2024) 'Towards autonomous cybersecurity: an intelligent AutoML framework for autonomous intrusion detection', in *Proceedings of the Workshop on Autonomous Cybersecurity (AutonomousCyber '24)*. New York: ACM. doi:10.1145/3689933.3690833. Available at: https://dl.acm.org/doi/epdf/10.1145/3689933.3690833 (Accessed: 13 March 2026).

[15] Canadian Institute for Cybersecurity (2017) *Intrusion detection evaluation dataset (CIC-IDS2017)*. Available at: https://www.unb.ca/cic/datasets/ids-2017.html (Accessed: 13 March 2026).

[16] Skrodelis, M. and Romanovs, A. (2026) 'Machine learning for critical infrastructure protection: challenges and opportunities', *Journal of Cybersecurity and Privacy*, 6(1), pp. 1–18. Available at: https://doi.org/10.3390/jcp6010001 (Accessed: 9 May 2026).

[17] H2O.ai (2024) H2O AutoML: automatic machine learning - H2O documentation, H2O.ai. Available at: https://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html (Accessed: 1 April 2026).

12. Appendices

Appendix A: CICIDS2017 Dataset Summary

Property	**Value**
File used	Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
Raw records	286,467
Clean records (after inf/NaN removal)	286,096
Features (after label removal)	77
Training set (80%)	228,876
Test set (20%)	57,220
Class 0 = BENIGN	127,292
Class 1 = PortScan	158,804

Appendix B: Full Python Implementation provided in the student cartridge